



Master's Degree Course in Computer Science

Master's Thesis

Lazy Declarative Heuristics in Answer Set Programming: Design and Evaluation of a clingo Propagator

Supervisor at the University of Calabria:
Prof. Carmine Dodaro

Candidate:
Pierpaolo Spadafora

Supervisors at the University of Potsdam:
Prof. Dr. Torsten Schaub
Dr. Javier Romero Dávila

Student ID: 263722

Contents

Abstract	iii
1 Introduction and Background	1
1.1 The Cost of a Dynamic Heuristic	1
1.2 Answer Set Programming	3
1.3 Ground-and-Solve: gringo, clasp, and Conflict-Driven Search	3
1.4 Domain-Specific Heuristics in clingo	4
1.5 Lazy Grounding, Alpha, and Heuristics over Partial Assignments	4
1.6 Extending clingo: Propagators and the Heuristic Interface	5
1.7 Objective and Contributions	5
2 Methodology	7
2.1 The Experimental Design: Two Axes, Four Variants	7
2.1.1 Simulating Alpha semantics	8
2.1.2 Methodological Rationale for the Exploratory Variants	10
2.1.3 Weights, Priorities, and Flattening	11
2.1.4 Problems on Bigger Weights	11
2.2 System Architecture	12
2.3 Translation of the Heuristic Directives	14
2.3.1 Lazy heuristic syntax for the C++ backend	14
2.3.2 Lazy heuristic syntax for the Prolog backend	17
2.3.3 What the Two Languages Cannot Express	18
3 Implementation and Benchmark Setup	20
3.1 Inside the Native Backend	20
3.1.1 Parser and Runtime Representation	20
3.1.2 Incremental Aggregates	20
3.1.3 Evaluation and Decision	21
3.2 The Prolog Backend	22
3.2.1 Rule Strings and Normalization	22
3.2.2 The Runtime Program	22
3.2.3 Synchronization and Decision	23
3.3 Benchmark Problems and Encodings	23
3.3.1 Balanced Sum Problem (BSP)	23
3.3.2 Partner Units Problem (PUP)	25
3.3.3 House Reconfiguration Problem (HRP)	25
3.3.4 Alpha as an External Reference	25

3.4	Benchmark Infrastructure and Metrics	26
4	Results	27
4.1	Experimental Setup and Dataset	27
4.1.1	Campaign Overview and Failure Modes	27
4.2	Reduction of the Ground Heuristic Component	29
4.3	BSP: Runtime Behaviour	31
4.4	BSP: Rules, Variables, and Memory	34
4.5	PUP: the Grounding Bottleneck at Scale	35
4.6	HRP: Guidance Without Dynamic Aggregates	39
4.7	An External Reference: Alpha	41
4.8	The clingo Reading of Partial Information: <code>gc</code> and <code>lc</code>	46
4.8.1	BSP: inertness	46
4.8.2	PUP: the same inertness, with the same surcharge	46
4.8.3	HRP: misguidance	47
4.8.4	What the three cases share	47
4.9	The Price of Laziness: Per-Decision Cost	48
4.10	Two Ground Simulations of Alpha: <code>ga</code> versus <code>ga_weak</code>	52
4.11	Evaluating Pre-Materialized Auxiliary Predicates	54
4.12	Effect of the Constraint Formulation	55
4.12.1	The full <code>_co</code> family: do the heuristics still matter?	56
4.13	C++ versus Prolog: Backend Comparison	58
4.14	Summary of Results	59
5	Discussion and Conclusions	63
5.1	Discussion of the Results	63
5.2	Limitations	68
5.3	Implications	69
5.4	Future Work	69
5.5	Conclusions	71
	Bibliography	73

Abstract

In the area of Answer Set Programming (ASP), ground-and-solve systems such as *clingo* require the instantiation of first-order rules before search begins. Although declarative domain-specific heuristics provide valuable search guidance, dynamic heuristics that depend on runtime search state (such as dynamic aggregates) suffer from combinatorial explosion during grounding. Conversely, lazy-grounding systems evaluate heuristics dynamically over partial assignments, but incur substantial search overhead because grounding and solving are interleaved throughout search. This thesis introduces a lazy declarative heuristic framework for *clingo* using its propagator interface, combining dynamic heuristic evaluation with the efficiency of ground-and-solve systems.

Two execution backends are provided: a compiled C++ backend with incremental aggregate state maintenance, and an embedded SWI-Prolog engine with relational query evaluation over the solver trail, which it reads in place through foreign predicates. Our methodology decouples heuristic representation (ground vs. lazy) from interpretation semantics (*clingo*-like vs. *Alpha*-like) through order-preserving weight flattening and the appropriate evaluation of default negation on unassigned atoms. We demonstrate that this semantic alignment is essential for dynamic heuristics to correctly steer the search. Benchmark results on three hard combinatorial problem families (the Balanced Sum Problem, the Partner Units Problem, and the House Reconfiguration Problem) show that the lazy propagator avoids grounding bottlenecks and drastically reduces memory consumption while preserving optimal search guidance. Compared to the lazy-grounding solver *Alpha*, our framework achieves the performance of ground-and-solve systems while maintaining the low memory cost of lazy-grounding solvers evaluating dynamic aggregates in heuristics.

Chapter 1

Introduction and Background

1.1 The Cost of a Dynamic Heuristic

Answer Set Programming (ASP) is a declarative approach to combinatorial search. Instead of describing an algorithm step by step, the modeler describes the properties that an admissible solution must satisfy, and a solver enumerates the assignments that satisfy them [1, 2, 3].

ASP solvers are essentially highly optimized backtracking, or more appropriately *backjumping*, engines. Modern ASP solving is dominated by two main paradigms: ground-and-solve and lazy grounding.

In ground-and-solve systems, such as clingo, first-order rules are instantiated over the problem domain before search starts, including the domain-specific heuristics that direct the solver’s search. This approach can be severely memory-intensive, with grounding bottlenecks manifesting in two main scenarios: as problem instances scale, and whenever domain-specific heuristics depend on dynamic search state, for instance through an aggregate over the atoms that are true *at that point of the search*. Because grounding precedes search, the grounder must enumerate every possible value the aggregate could take, and a single heuristic ends up producing a number of ground objects that is combinatorial in the size of the instance. In contrast, lazy-grounding systems interleave grounding and solving. While this yields significantly higher memory efficiency, allowing a further reach on memory-intensive problems, it comes at the cost of a significantly slower search.

Depending on its underlying architecture, an ASP solver can be seen as an engine that compiles or interprets a very convenient and elegant declarative language [4] to search for a solution. The convenience of that language is paid for in control: the modeler states *what* an admissible solution is, but not *how* the engine should look for one. A pure backtracking approach behaves as an uninformed search, and on problems whose structure is known by construction, guiding the solver with that knowledge is extremely helpful.

Domain-specific heuristics are the means by which the modeler feeds that knowledge into the search. Syntactically they are declarative annotations that the grounder instantiates like ordinary rules, and their purpose is to tell the solver which atom to decide next, and with which polarity; their impact on solving is well documented [5, 6]. They are, however, part of the program, and clingo instantiates

the program before search starts. A heuristic is therefore expanded, in full, before the first decision is taken. That expansion can be relatively harmless as long as the heuristic depends only on the input. It stops being harmless as soon as the heuristic depends on *how the search is going*, which is precisely when a heuristic is most useful. Consider the directive used throughout this thesis to guide the Balanced Sum Problem (BSP), the problem of splitting $\{1, \dots, n\}$ into two sets $b/1$ and $c/1$ of nearly equal sum:

```
1 #heuristic b(X) : x(X), not c(X), S = #sum{Y : c(Y)}, W = ((n+1)*S)+X. [W,
  ↪ true]
```

The rule reads: prefer to place the element X into b , the more strongly the larger the sum S of what is already in c . The aggregate value S appears *inside the weight*, and the atoms $c(Y)$ it sums over are guessed during search rather than given as input facts. Since the grounder cannot know what S will be at decision time, it must enumerate every potential value of the sum ($S \in \{0, \dots, \frac{n(n+1)}{2}\}$) and instantiate a separate ground directive for every pair (X, S) . This yields $\Theta(n^3)$ ground objects (over one million, 1,010,200, directives at $n = 100$) for the symmetric rules of $b/1$ and $c/1$. All of them instantiate the same single rule the modeler wrote, and all but the one matching the current partial sum are inert at any given decision. A domain-specific heuristic introduced to help the engine solve a problem quicker ends up putting the answer out of reach.

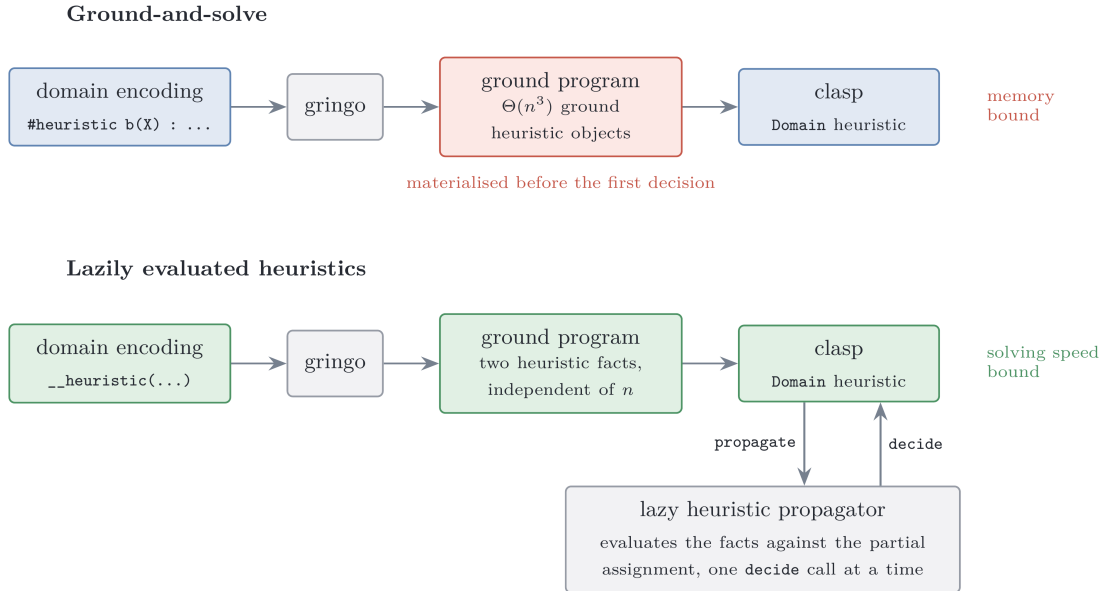


Figure 1.1: The same heuristic under the two approaches compared in this thesis.

The alternative is to leave the heuristic symbolic and to evaluate it while the search runs, when the partial assignment that S refers to actually exists. This is the natural approach in lazy-grounding solvers such as Alpha [7, 8, 9].

1.2 Answer Set Programming

A *normal rule* in ASP has the form

$$a \leftarrow b_1, \dots, b_m, \text{ not } c_1, \dots, \text{ not } c_n,$$

where a , the b_i and the c_j are atoms: the head a is derived whenever every positive body atom b_i is derivable and no negative body atom c_j holds. The negation *not* is *default negation*: *not* c holds in the absence of a derivation for c . Because a rule read this way can justify itself, the meaning of a program is not given by the rules alone but through the notion of a stable model. Fix a set of atoms X , the candidate solution, and a ground program P . The *reduct* P^X is obtained by deleting every rule whose negative body intersects X , and by deleting the negative literals from the rules that survive. The reduct is a positive program, so it has a unique least model, and X is a *stable model* (or *answer set*) of P exactly when it coincides with that least model [1]. On the two-rule program

$$\begin{cases} a \leftarrow \text{not } b \\ b \leftarrow \text{not } a \end{cases}$$

the candidate $X = \{a\}$ gives the reduct $\{a \leftarrow\}$, whose least model is $\{a\}$: stable. The candidate $\{a, b\}$ deletes both rules, leaving the least model $\emptyset$: not stable. A stable model is then a set of atoms both closed under the rules and justified by them, in which every atom has a non-circular derivation that survives the model's own assumptions about what is false.

In the *guess-and-check* idiom a choice construct opens the solution space, integrity constraints prune the candidates that violate the specification, and the answer sets of the program correspond one-to-one to the solutions of the problem. ASP syntax extends normal rules with first-order variables, arithmetic, conditional literals, and three constructs this thesis uses throughout. A *choice rule* such as $\{ \text{b}(X) \} \text{ :- } \text{x}(X)$ leaves the truth values of its head atoms unconstrained, making these *choice atoms* the explicit decision points of the solver. An *integrity constraint* :- Body is a rule with empty head and forbids every model that satisfies *Body*. An *aggregate atom* such as $\#\text{sum}$, $\#\text{count}$, $\#\text{min}$ or $\#\text{max}$ condenses a set of weighted contributions into a single comparable value [3]. Aggregates are central to this work: the heuristics under study rank decisions by expressions such as “the current sum of the elements already placed in a set”, and how and *when* such an aggregate is evaluated is exactly what differentiates the systems compared here.

1.3 Ground-and-Solve: gringo, clasp, and Conflict-Driven Search

clingo [3] couples the grounder gringo with the solver clasp into a single system. The grounder instantiates the first-order program over its Herbrand universe, producing a variable-free program. To mitigate combinatorial explosion, gringo restricts this expansion to rule instances whose positive body is potentially derivable; as a result,

the ground program is generally much smaller than a full instantiation, though it must still be materialized in full before search starts. The solver clasp [10] then searches for the stable models of the ground program using *conflict-driven nogood learning* (CDNL), an algorithm that learns from conflict-inducing choices. The solver maintains a *partial assignment*, a consistent set of literals over the program atoms; it alternates *unit propagation* steps with *decision* steps, and when propagation derives a contradiction, it analyzes the conflict, learns a nogood built around the *First Unique Implication Point* (1UIP), and *backjumps* to the decision level at which that nogood becomes unit. Which atom to decide on, and with which polarity, is delegated to a *decision heuristic*; clasp’s default is activity-based, in the family introduced by Moskewicz et al. [11], and favors atoms that appear in recent conflicts.

This approach entails that every atom and every rule clasp will ever see must exist before the first decision is made and also that the size of the ground program is the dominant memory cost of the pipeline. The *grounding bottleneck* is a well-known limitation of ground-and-solve systems [8]; Chapter 4 measures one concrete manifestation of it, namely heuristic directives whose weight depends on an aggregate value.

1.4 Domain-Specific Heuristics in clingo

clasp allows the modeler to steer its decision heuristic using domain-specific knowledge through the `#heuristic` directive [5]:

```
1 #heuristic A : Body. [W@P, M]
```

Here, A is the target domain atom whose decision choice is steered according to the modifier M (e.g., setting its sign or priority level). The heuristic applies only if the rule *Body* is satisfied. If multiple heuristic directives target the same atom A , clingo selects the one with the highest priority P to determine the atom’s modifier and weight. Across distinct target atoms, clasp then prioritizes candidates with higher weight W . In contrast, Alpha treats (P, W) as a global lexicographic rank over all atoms (Section 1.5).

1.5 Lazy Grounding, Alpha, and Heuristics over Partial Assignments

Alpha is the reference state-of-the-art lazy-grounding solver for this work.

Comptoi-Taupe et al. extended Alpha with declaratively specified domain-specific heuristics evaluated against the *partial* assignment maintained during solving [8]. Its semantics differs from clingo’s on two main points.

Specifically, Alpha evaluates negated body literals against unassigned atoms and prioritizes P over weight W in the annotation $[W@P]$, always preferring higher priority values. Two atoms are compared by weight only when they share the same

priority. A complete formalization of these semantic differences and their integration into our experimental design is provided in Chapter 2.

1.6 Extending clingo: Propagators and the Heuristic Interface

Standard clingo does not evaluate heuristic conditions dynamically at search time. However, it provides an extension mechanism: *theory propagators*, user-defined components registered with the solver that participate directly in the search loop [12]. This thesis leverages that interface to provide search guidance computed dynamically on the trail rather than fully materialized before search.

A propagator has four callbacks. During `init`, it inspects the symbolic atoms of the ground program, maps the atoms it cares about to solver literals, and registers *watches* on them. During search, `propagate` is invoked when a watched literal becomes true, `undo` when backtracking retracts it, and `check` on total assignments. The propagator is invoked by clasp only when a watched literal changes state.

The `clingo::Heuristic` interface inherits from `Propagator` and extends it with a `decide` callback, invoked at each decision step *before* the solver’s built-in heuristic. The `Heuristic` object receives the current assignment of the solver and either returns a signed literal, which becomes the next decision, or returns 0, in which case clasp falls back to its default activity-based heuristic (VSIDS [11]). This is the mechanism the rest of the thesis builds on. It keeps the declarative heuristic descriptions in memory, evaluates them incrementally against the trail, and answers each `decide` call with the best applicable target.

1.7 Objective and Contributions

This work asks whether the same effect of lazily evaluating heuristics can be obtained inside clingo without giving up its ground-and-solve pipeline and speed advantage, that is, whether a ground-and-solve engine can be equipped with lazily evaluated heuristics (Figure 1.1) and whether the overhead of dynamic evaluation still allows the engine to outperform a state-of-the-art lazy-grounding solver.

The thesis makes the following contributions.

1. **A lazy heuristic evaluator for clingo:** A propagator, registered through the public `clingo::Heuristic` interface, that keeps a declarative heuristic in memory as a compact description and evaluates it at every decision against the current partial assignment.
2. **Two propagator backends:** A compiled C++ backend, which materializes one candidate per ground target and maintains incremental aggregate states, and an in-process SWI-Prolog backend, which evaluates the heuristic as an ordinary logic program against the solver trail, read directly through foreign predicates.
3. **Semantics and ablation study:** Ground-and-Solve, Lazy, clingo semantics and Alpha semantics combined give four variants of every encoding plus a heuristic-

free baseline, so that no measured difference has to be attributed to two causes at once.

4. **Weight translation from Alpha’s syntax:** Alpha’s priority levels are global, while clingo’s are local to a single atom. Section 2.1.3 presents the transformation that folds a level into the weight, the conditions under which it preserves the intended order and the weight field limit imposed by clingo.
5. **Trade-off analysis of the lazy approach:** By collecting per-decision metrics in both backends, we evaluate the exact computational overhead of lazy grounding against its savings in program size.

The evaluation covers three problem families, the Balanced Sum Problem (BSP), the Partner Units Problem (PUP), and the House Reconfiguration Problem (HRP), on both backends. The implementation is publicly available.¹

¹<https://github.com/pierspad/clingo-lazy-heuristic-evaluator>

Chapter 2

Methodology

2.1 The Experimental Design: Two Axes, Four Variants

Every experiment in this thesis compares encodings of the same problem that differ in how the domain-specific heuristic is represented and interpreted. The rules that define the problem, its guess and its constraints, are held fixed across the variants of a family; what varies are the two properties that Section 1.4 and Section 1.5 identified as the points on which clingo and Alpha disagree, and they vary independently.

The *approach* axis fixes where the heuristic is evaluated. In the ground-and-solve approach the heuristic is written with native `#heuristic` directives: gringo expands them before search and clasp’s Domain heuristic consumes the resulting ground objects. In the lazy approach the heuristic remains a handful of ground facts, `__heuristic(...)` for the native backend and `heuristic("...")` for the Prolog one, which the propagator reads at its initialization and evaluates at every decision step against the partial assignment.

The *semantics* axis fixes how a heuristic condition reads partial information: the clingo-like semantics, under which `not a` holds only once *a* has been assigned false and an aggregate has a value only once its sources are determined, against the Alpha-like semantics, under which `not a` holds as soon as *a* is not assigned true, and an aggregate is computed over the atoms currently true. Only the second reading makes a heuristic dynamic in the sense of reacting to a partial assignment during search, as motivated in Section 1.1: an instruction such as “prefer the element whose insertion keeps the two sums balanced” presupposes that the sum can be evaluated against an assignment that is still being built; under clingo’s semantics, that sum simply has no value until it is too late for the guidance to matter.

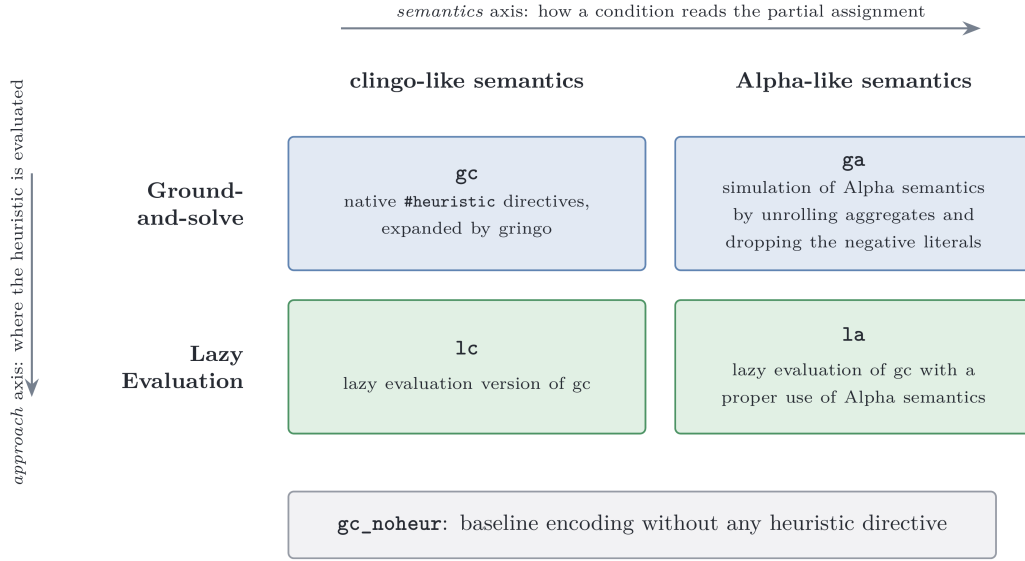


Figure 2.1: The experimental matrix. Prefixes name the approach, suffixes the semantics.

Crossing the two axes yields the four variants that the rest of the thesis refers to as: **gc** and **ga** for the ground approach under the clingo and the Alpha semantics, **lc** and **la** for the lazily evaluated one, the prefix naming the approach and the suffix the semantics. To them is added the heuristic-free baseline **gc_noheur**, which carries the same domain logic with no heuristic at all and measures what the solver does when it is left alone (Figure 2.1).

To isolate specific architectural and modeling mechanisms beyond this primary matrix, the experimental design also incorporates two exploratory variants, **ga_weak** and **la_aux**, and a second copy of the entire BSP matrix, the **_co** family, in which the domain constraint is formulated in a different way. Their empirical motivation and methodological role in disentangling the contributions of default negation, body materialization, and constraint formulation are discussed in Section 2.1.2.

2.1.1 Simulating Alpha semantics

Reproducing an Alpha-like behavior within clingo requires addressing two distinct aspects: default negation over unassigned literals and dynamic aggregate evaluation.

Default negation over unassigned literals

The first transformation concerns negative literals in heuristic bodies: `not c(X)` in BSP, `not not_assigned_sensor_unit(S,U)` in PUP, and the guards `not fullCabinet(C)` and `not assignedThing(T)` in HRP.

Clasp applies heuristic modifications only to variables that are currently unassigned, as enforced in `DomainHeuristic::propagate`:

```
1  if (s.value(a.var) == value_free && a.prio >= prio) { applyAction(s, a, prio);
    ↪ }
```

It follows that *a directive whose body entails the truth value of its own target atom can never fire*: once the body conditions hold, the target is already decided, and clasp skips it.

All heuristic directives in the suite that test the search state through negative literals show the same problem:

- In **BSP**, `b(X)` and `c(X)` are complementary choices, so the condition `not b(X)` is satisfied only once the target `c(X)` is already derived.
- In **PUP**, the heuristic body repeats the negative literal of the rule that derives the target atom, namely `not not_assigned_sensor_unit(S,U)`.
- In **HRP**, the placement directives for `cabinetT0thing(C,T)` require the guard `not assignedThing(T)`; in ground clingo this holds only when the candidate assignments for T are already proven false, meaning the target is never unassigned (`value_free`) when the directive evaluates.

Consequently, none of these directives ever fires in the `gc` variant. In HRP the same encoding also carries directives with no condition at all, which are applied at initialization and do steer the search; the conditional ones listed above stay inert.

The ground variant `ga` therefore drops these negative literals from the heuristic bodies. For directives where the negated literal simply tests the target atom or its complement (as in BSP and PUP), this simplification is completely safe: nothing is lost that could have fired, and clasp’s `value_free` check still prevents modifying an already assigned atom. In HRP, however, the dropped literals also include capacity guards such as `not fullCabinet(C)`, which describe a shared resource rather than the target atom itself. There, dropping the guard is a necessary static approximation: it enables the directive to fire on unassigned atoms from the start of search, at the price of never deactivating once a cabinet becomes full.

Deleting `not a` from a heuristic body preserves the Alpha reading exactly when a is the target atom or its complement, and only then: in that case the deleted guard duplicates a test clasp performs anyway, since a directive is applied only to a `value_free` variable. When a is any other atom, the guard carries information the solver does not re-derive, and removing it changes which decisions the heuristic proposes.

Dynamic aggregates by bound-guarded unrolling

The second transformation replaces dynamic aggregate equality with bound-guarded unrolling over all candidate values. For BSP:

```

1  sum_value(0..tot).
2  #heuristic b(X) : x(X), sum_value(S), S <= #sum{Y : c(Y)},
3      W = ((n+1)*S)+X. [W, true]
```

Here the directive is unrolled over all `sum_value(S)` and guarded by a bound condition, so that during search the currently satisfied bounds encode the current aggregate value, provided the guard is monotone, i.e. once satisfied it stays satisfied as further atoms become true.

Without explicit priorities all unrolled directives share the default priority 0. Clasp applies newly satisfied directives by overwriting previous ones; it therefore retains the *most recently activated* directive, not the maximum-weight one. To match the reference heuristic, the unrolling must align the activation order with the weight order:

- **Lower-bound guards** ($V \leq f$): As f increases, the active set $\{V : V \leq f_{curr}\}$ expands upwards. The last activated value is $V = f_{curr}$, which yields the maximum weight if $W(V)$ is strictly *increasing* (e.g. in BSP, where $W(S) = (n+1)S + X$).
- **Upper-bound guards** ($V \geq f$): The active set $\{V : V \geq f_{curr}\}$ instead expands downwards, as f decreases, and the alignment requires $W(V)$ to be strictly *decreasing*.

2.1.2 Methodological Rationale for the Exploratory Variants

While the four main variants evaluate the core matrix of approach versus semantics, the following are introduced as experimental controls to isolate specific factors:

Isolating Alpha-Negation from Dynamic Aggregates (ga_weak). Alpha-like semantics treat default negation as satisfied over unassigned literals, and evaluate aggregates dynamically over currently true atoms. While **ga** simulates both via extensive ground unrolling, **ga_weak** acts as an ablation study that applies Alpha’s negation semantics while avoiding unrolling aggregates. This investigates whether adapting default negation alone provides any improvement during search, or whether dynamic aggregate evaluation is strictly indispensable for effective search guidance.

Fewer directives, or less grounding? (la_aux). The lazy variants differ from the ground ones in two ways at once: the grounder emits far fewer heuristic-related objects, and the evaluation of the heuristic conditions stays out of the ground program altogether. The **la_aux** variant separates the two effects. It keeps the lazy propagator, but writes the conditions of the BSP heuristic as ordinary ASP rules, one atom **h_b**(X,S) for each element X and each candidate sum S , which the propagator then queries at decision time. The heuristic directives stay few, while the pre-evaluation goes back to the grounding phase; the variant therefore shows which of the two is responsible for the gain.

Disentangling Heuristic and Problem Constraint Bottlenecks (the _co family). In the baseline BSP encoding, the balance requirement is enforced by computing the sums of both partitions independently and verifying that their difference does not exceed one via an integrity constraint (`:- sumB(SB), sumC(SC), (SB - SC)**2 > 1`). Because the possible partition sums scale quadratically with n , joining all pairs of candidate sums during grounding triggers a severe combinatorial explosion (materializing over 52.8 million propositional rules at $n = 120$). Consequently, total runtime remains entirely dominated by grounding the base problem

constraints. This imposes an artificial performance floor across all variants, masking the true computational footprint of the heuristic layer.

The compact (`_co`) transformation resolves this bottleneck by exploiting the invariant that the total sum of all elements is known a priori ($tot = \sum_{i=1}^n i$). Instead of generating auxiliary predicates for both sums and checking them quadratically, the balance condition is restated as a single aggregate constraint with constant interval bounds, requiring the sum of partition b to fall strictly within $[[tot/2], [tot/2]]$ (Section 3.3.1). This removes the cross-product join over candidate sums entirely, collapsing the base encoding by five orders of magnitude (from millions of rules to just a few hundred). As an experimental control, the `_co` family eliminates the base grounding overhead, cleanly isolating the representational cost of the heuristics and clarifying whether dynamic guidance remains beneficial when domain propagation is already efficient.

2.1.3 Weights, Priorities, and Flattening

Section 1.5 recorded that the two systems read the annotation $[W@P]$ of a heuristic differently. In Alpha, the pair is an *absolute* rank over all heuristic instances, read lexicographically, with P compared first: the priority P is a global level and decides first, so no instance of level P is considered while an instance of a level $P' > P$ is still applicable, and the weight W only orders the instances that share the same level. In clingo, the evaluation is different: P is local, arbitrating only between heuristics relative to the *same* target atom, while the order in which distinct atoms are decided comes from the weight alone. An encoding written for Alpha therefore cannot be handed to clingo verbatim.

The intended order can nevertheless be recovered, by *flattening* the level into the weight:

$$W_{flat} = W + P \cdot M, \quad M > \max W - \min W,$$

where the intra-level weights W range over $[\min W, \max W]$ and the level multiplier M is larger than that range. Under this condition $W + P \cdot M > W' + P' \cdot M$ holds whenever $P > P'$, regardless of the two individual weights, and reduces to $W > W'$ when $P = P'$: the total order induced on the atoms is then the one Alpha’s absolute levels induce. When all weights are non-negative, which is the case in every encoding of this suite, $M > \max W$ suffices.

Each benchmark fixes its own M . In the PUP encodings M is the constant offset 100,000 added to the zone weights, against a maximum sensor weight of 26,220 on the benchmark instances; in HRP M is derived from the instance as `levelMul(LM) :- LM = MC + MR + 10` over the maximum cabinet and room identifiers, which gives 16 on the smallest instance and 54 on the largest. BSP does not need a multiplier, as discussed in Section 3.3.1.

2.1.4 Problems on Bigger Weights

clasp stores the weight of a `#heuristic` modification in a signed 16-bit integer field and clamps any value exceeding 32,767. Consequently, two ground directives whose weights both exceed 32,767 become indistinguishable to clasp’s Domain heuristic. A

flattened weight must therefore satisfy $W + P \cdot M \leq 32,767$ for the ground variants to rank as intended. HRP stays far below this bound; PUP exceeds it on the zone directives, with the consequences discussed in Section 4.5; and BSP, whose maximum weight grows as $\Theta(n^3)$, crosses it within the benchmarked range. What matters there is not the largest weight the grounder emits, but the largest one the search actually reaches: directives whose aggregate value is never attained may clamp without any observable effect. On BSP that effective crossing occurs at $n = 51$, as analyzed in detail in Section 4.10.

In the compact `_co` encoding (Section 4.12.1), weights saturate in the exact same way, yet runtime is unaffected because propagation resolves choices directly without needing heuristic guidance. The lazy backends avoid this limitation altogether. Rather than assigning static weights to clasp, the propagator ranks candidate targets dynamically at runtime and directly returns the chosen decision literal.

The suite adopts clingo’s convention in *all four variants and in both backends*: absolute levels always live in the weight, simplifying the propagator as it does not need to account for a separate priority field. Once global levels are folded into weights, weight becomes the sole ordering criterion under both semantics. Accordingly, the native lazy language (Section 2.3) omits the priority field entirely.

A last convention concerns empty aggregates: `#max` over an empty set is 0 in Alpha but $-\infty$ (`#inf`) in clingo. All encodings therefore write `#max{0; ...}` with an explicit neutral element, and both lazy backends implement the same rule (the native `MaxState` returns 0 when empty; the Prolog runtime defines `dyn_max` with a 0 default).

2.2 System Architecture

The project maintains two modified copies of clingo. They share the same upstream sources and differ only in their extensions: the build in `clingo-native` carries the C++ backend, while the build in `clingo-prolog` carries the Prolog backend, configured with `clingo_USE_SWIPL=ON` so that the SWI-Prolog engine is linked in-process, making its memory impact easier to account for. In both builds, the extension lives entirely under `libclingo`, exposes the same class name, and is registered at the same point of `clingo_app.cc`:

```

1 HeuristicPropagator my_propagator;
2 clt.register_propagator(my_propagator, true);

```

The propagator is registered on *every* run of the modified binary. On a program that contains no lazy heuristic facts, the propagator itself is inactive: `init` finds nothing to watch and `decide` always returns 0, so every decision is left to clasp’s own branching heuristic. The same binary therefore also runs the ground variants (with clingo’s standard `#heuristics`) and the heuristic-free baseline without requiring a separate build.

The second argument, `true`, instructs clasp to call the propagator sequentially, avoiding race conditions on the incremental state described in Section 3.1.1. No

other parts of clingo are modified. Figure 2.2 shows the architecture of the two binary build variants.

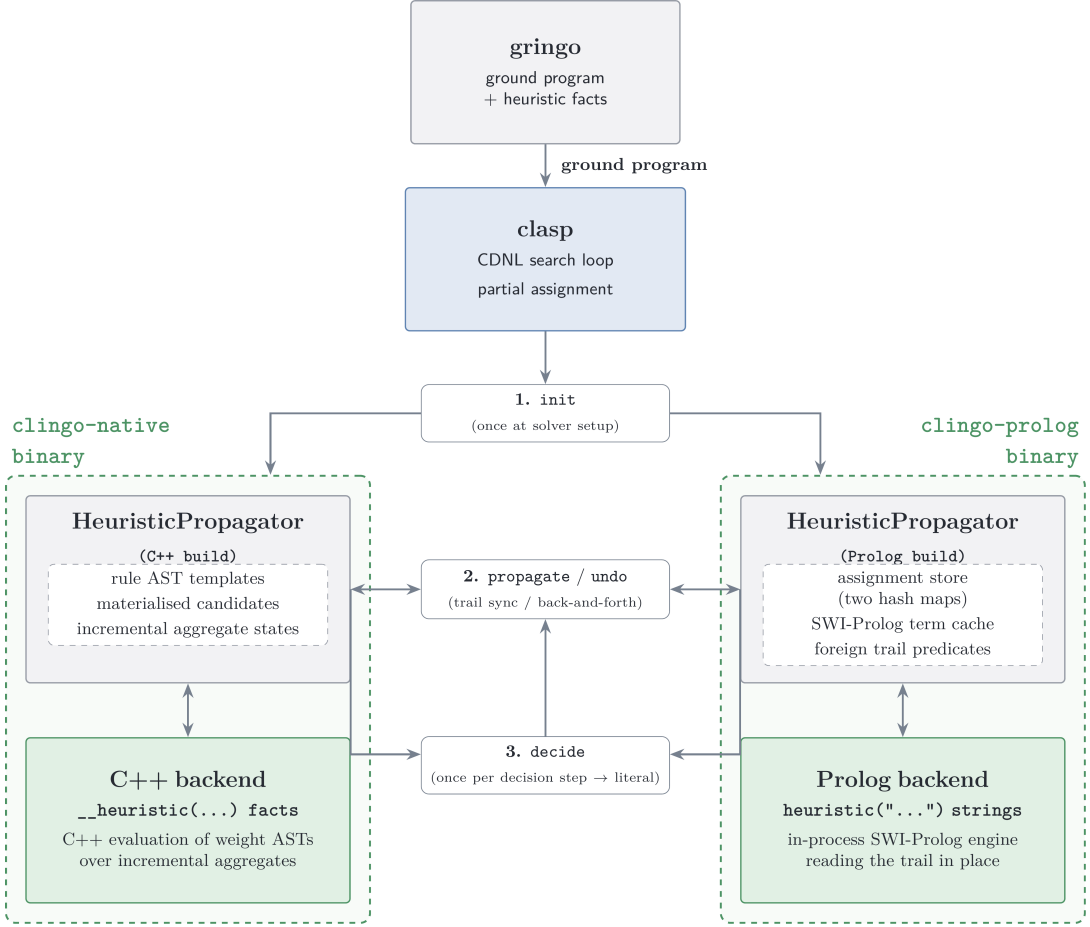


Figure 2.2: System architecture of the two binary variants, showing their respective heuristic propagators and evaluation backends attached to clasp.

In both builds, the class implements `clingo::Heuristic`, the propagator interface extended with the `decide` callback (Section 1.6). Both backends follow the same lifecycle, built on `init`, `propagate`, `undo` and `decide`:

- During `init`, they scan symbolic atoms for trigger facts (`__heuristic(...)` in the C++ build and `heuristic("...")` in the Prolog build), construct their runtime representation, and register watches on solver literals.
- During forward search, `propagate` updates the internal state of the aggregates appearing in the heuristic bodies, together with the candidate states, as watched literals become true.
- During backtracking, `undo` restores those aggregate and candidate states as literals are retracted from the trail.
- At each decision point, `decide` either returns a signed literal for the highest-ranked target or returns 0, delegating the decision to clasp’s fallback heuristic.

The two backends are not symmetric in what they can express. The native backend maintains an incremental state for `#sum`, `#count`, `#min` and `#max` over one predicate and one argument position, and can only filter the aggregated set by matching its arguments with those of the target atom; the Prolog backend evaluates an arbitrary `aggregate_all/3` goal and is unrestricted in this respect. Section 2.3 states the fragment each backend accepts, and Section 2.3.3 what falls outside it.

To activate this mechanism, runs are launched with the flag `-heuristic=Domain`. A detailed discussion of the architectural differences between the two backends is provided in Chapter 3.

The propagator never adds a constraint, never derives a literal, and never refuses one. It only reorders the decisions clasp would have taken anyway, so the answer sets are untouched, and a heuristic can make a run slower or faster but cannot change the meaning of the encoding (detailed in Section 3.1.3).

2.3 Translation of the Heuristic Directives

Neither backend reads native `#heuristic` directives directly. Instead, a custom declarative language replaces each directive with a single ground fact that *describes* the heuristic rule.

This substitution is driven by two main reasons:

- Gringo treats a `#heuristic` directive as an ordinary ASP rule and immediately grounds it. By the time the propagator initializes in `init`, the combinatorial explosion has already occurred. The propagator needs the high-level *template* which gringo does not ground, clingo C++ API does not expose but the propagator can read.
- Aggregates must survive grounding unevaluated so they can be monitored dynamically during search. Standard ASP syntax provides no such mechanism: an aggregate inside a rule body is either evaluated statically by the grounder or unrolled over its value domain.

The lazy fact is therefore the original directive rewritten in a form that passes through the grounder untouched. Its structure mirrors the original construct-by-construct. The remainder of this section details this representation. We introduce each construct by rewriting the BSP heuristic (Section 1.1) step by step. Finally, Section 2.3.3 outlines the expressive limitations of both backends.

2.3.1 Lazy heuristic syntax for the C++ backend

```
1 #heuristic b(X) : x(X), not c(X), S = #sum{Y : c(Y)}, W = ((n+1)*S)+X. [W,  
   ↪ true]
```

Target. The target atom is `b(X)`, where `X` ranges over the ground instances. The lazy fact specifies only the predicate name, `__target(b)`. During `init`, the propagator queries clingo’s atom table for all ground instances of `b(...)` already materialized by gringo.

Positive body. The condition `x(X)` shares its argument with the target: the candidate for `b(7)` is alive only while `x(7)` is true. A condition of this shape is written as the bare predicate name, `x`, and the propagator matches its arguments against the target tuple position by position.

However, a body predicate may have a different arity; its arguments may sit in different positions; or a value may have to be read out of it and used in the weight. The correspondence is then stated explicitly:

```
1  __body(pred, __match(src, tgt), __bind_arg(var, idx))
```

`__match(src, tgt)` requires the argument at the `src` position of the body atom to equal the argument at the `tgt` position of the target atom, where both `src` and `tgt` are integers. All indices are 0-based, and at least one `__match` is mandatory. `__bind_arg(var, idx)` extracts argument `idx` of the body atom into a variable.

Given the ground facts:

```
1  score(1,10). score(2,5). score(3,7).
```

and the target atoms:

```
1  choose(1). choose(2). choose(3).
```

the directive:

```
1  __heuristic(__target(choose), __body(score, __match(0,0), __bind_arg(w,1)),
  ↪  __weight(w), __modifier(true)).
```

pairs each `choose(X)` with the `score(X,w)` that agrees with it on the first argument, and binds `w` to the second argument of that atom. The weights are then 10, 5 and 7, so the propagator decides `choose(1)` first.

Negative body. The condition `not c(X)` likewise matches its arguments against the target tuple. Negative body literals are specified by prefixing the predicate name with `__n_`.

Semantics. The reading of that negation is fixed by the `__semantics(S)` selector, which decides whether `__n_c` means “`c(X)` is assigned false” or “`c(X)` is not assigned true”. The same selector fixes the reading of aggregates, described below. It takes `alpha` or `clingo`, and defaults to `clingo`, so that a directive written without it behaves as the native `#heuristic` it replaces.

Aggregates. Aggregates are evaluated lazily during search. For instance, `--bind(s, --sum(c))` binds variable `s` to the sum over the predicate `c` (by default aggregating its first argument). The operators `--count`, `--min`, and `--max` follow the same structure.

An aggregate can be restricted using `--filter(src_idx, tgt_idx, offset)`, which narrows it *per target*, keeping only source atoms whose argument at position `src_idx` equals argument `tgt_idx` of the target plus an optional `offset`. For instance, in PUP, retrieving the maximum number of sensors on the *previous* unit ($U - 1$) for a target `assigned_zone_unit(Z,U)` is expressed as:

```
1  --bind(m1, --max(num_sensors_on_unit, 0, --filter(1, 1, -1)))
```

where 0 selects the aggregated argument N in `num_sensors_on_unit(N, U')`, and the filter matches $U' = U - 1$.

Weight. Weight is expressed as a *term tree* such as:

```
1  --weight(--add(--mul(s, B), self))
```

which represents $(s \cdot B + self)$, where `self` is a shorthand for the target atom's argument at index 0 without requiring a variable binding.

The binary constructors `--add`, `--sub`, and `--mul` form expression trees whose leaves belong to three categories: integer constants, variables of the heuristic language, and target arguments (accessed through `self` or `--bind_target_arg(var, idx)` for any other).

At `init`, the propagator compiles all weight term trees into ASTs. During search, each active heuristic object evaluates its weight AST dynamically against the current trail state, avoiding the memory overhead of materializing them in the ground program. Only candidates with a strictly positive resolved weight are taken into account; the others are ignored, leaving their decision to the solver's default heuristic.

Modifier. The `--modifier(M)` selector specifies which of clingo's heuristic modifiers to use: `level` sets the ranking value alone, `sign` sets the preferred polarity alone, while `true` (the default) and `false` set both.

The solver's `decide` interface expects a *signed* literal, so a ranking value alone does not determine a decision. Each target keeps one slot per channel, ranking value and preferred polarity, and when several directives write to the same slot the one with the highest weight wins. This is what replaces clingo's local priority. Across the benchmarks all directives use `true`, except for the final layer of HRP, which uses `false` to prune the remaining choice atoms.

Assembling the components gives the fact in full:

```
1  --heuristic(--target(b),
2      x,
3      --n_c,
4      --bind(s, --sum(c)),
```

```

5      __weight(__add(__mul(s, B), self)),
6      __semantics(alpha),
7      __modifier(true))
8      :- B = n + 1.

```

2.3.2 Lazy heuristic syntax for the Prolog backend

Unlike the native backend, which only supports a fixed set of predefined constructs, the Prolog backend imposes no syntactic restrictions. A heuristic is declared as a string inside a `heuristic("...")` fact and parsed directly as a standard Prolog clause; consequently, its body accepts any goal that the engine can prove.

```

1  heuristic(b(X), W, 0, true) :-
2      holds(x(X)),
3      target_available(b(X)),
4      alpha_not(c(X)),
5      aggregate_all(sum(Y), holds(c(Y)), S),
6      holds(heuristic_base(Base)),
7      W is Base * S + X.

```

The rule is structured into two main parts: the **head** encodes the four arguments of the `#heuristic` directive, the target atom (A), the calculated weight (W), the priority level (P), which is always 0 because global levels are folded into the weight (Section 2.1.4), and the modifier (M), while the **body** specifies the conditions and computations needed to derive them.

To interact with the solver's search state, the backend provides a few dedicated interface predicates: `holds/1` succeeds on atoms currently true on the trail and on static ground facts; `alpha_not/1` and `clingo_not/1` implement the two readings of default negation described in Section 1.5; and `target_available/1` discards candidates that are already assigned.

Everything else uses standard Prolog: arithmetic relies on `is/2`, and set aggregations use `aggregate_all/3`. For convenience, the backend provides the `dyn_*` wrappers (`dyn_sum/3`, `dyn_count/2`, `dyn_min/3`, and `dyn_max/3`); in particular, when no matching atoms are yet assigned on the trail (such as at the beginning of search), `dyn_min/3` and `dyn_max/3` safely return 0 instead of failing, allowing the heuristic to compute a weight and guide early decisions.

Table 2.1 summarizes the correspondence between the three notations.

construct	clingo directive	native lazy fact	Prolog rule string
<i>structure</i>			
target	b(X)	<code>--target(b)</code>	<code>heuristic(b(X), W, P, M)</code>
positive body, target tuple	x(X)	x	<code>holds(x(X))</code>
positive body, join	p(X,V)	<code>--body(p, --match(0,0), --bind_arg(v,1))</code>	<code>holds(p(X,V))</code>
negation	not c(X)	<code>--n_c</code>	<code>alpha_not(c(X)) clingo_not(c(X))</code>
aggregate	S = #sum{Y : c(Y)}	<code>--bind(s, --sum(c)) also --count, --min, --max</code>	<code>aggregate_all(sum(Y), holds(c(Y)), S)</code>
filtered aggregate	M = #max{N : q(N,U-1)}	<code>--bind(m, --max(q, 0, --filter(1,1,-1)))</code>	<code>U1 is U-1, dyn_max(holds(q(N,U1)), N, M)</code>
target arguments	X, Y in b(X,Y)	<code>self, --bind_target_arg(y, 1)</code>	ordinary head variables
<i>annotations</i>			
weight	W = ((n+1)*S)+X	<code>--weight(--add(--mul(s,B), self))</code>	W is B*S+X
modifier	[W, true]	<code>--modifier(true), default true</code>	fourth head argument
semantics	(fixed by the system)	<code>--semantics(alpha) --semantics(clingo)</code>	choice of <code>alpha_not</code> / <code>clingo_not</code>
target still free	(implicit in clasp)	implicit: an assigned target is skipped	<code>target_available(b(X))</code>

Table 2.1: The same heuristic in the three notations.

2.3.3 What the Two Languages Cannot Express

While Table 2.1 summarizes the syntax supported by each backend, the two engines do not cover the same subset of clingo’s heuristic language, nor does either support standard ASP heuristics in full. Instead, they reflect a clear design trade-off between evaluation speed and relational expressiveness.

The C++ fragment. The C++ backend is designed for high performance through incremental state tracking, but this restricts its language to a minimal template. Directives must target entire predicates rather than individual ground atoms, and arguments are strictly limited to numeric tuples.

In rule bodies, literals can only link directly to the target tuple: arbitrary joins between body atoms (such as $p(x,v)$, $q(v,w)$) and relational comparisons ($N < M$) are unsupported, requiring precomputation via auxiliary ASP rules. Similarly, aggregates operate strictly over a single predicate and argument, without support for internal joins or conditions.

Finally, weight arithmetic is restricted to basic operations ($+$, $-$, $\cdot$) yielding strictly positive values, and modifiers altering internal VSIDS scores (`init`, `factor`) are rejected. Because the backend also omits an explicit priority field, multi-level priority hierarchies must be flattened into weights (Section 2.1.3).

The Prolog fragment. In contrast, the Prolog backend prioritizes expressive flexibility. Because rule bodies are standard Prolog goals, arbitrary joins, comparisons, nested aggregations (via `aggregate_all/3`), and non-numeric terms are supported natively. Its head also includes an explicit priority argument (P, W) , directly preserving Alpha’s two-dimensional priority scheme without weight flattening. The primary limitation is that Prolog goals may generate terms that do not exist in the ground program, requiring the backend to filter out invalid candidates against clasp’s atom tables (Section 3.2.3). Unrestricted goals are re-evaluated from scratch at every decision point, introducing runtime overhead that is evaluated in Section 4.9.

Chapter 3

Implementation and Benchmark Setup

3.1 Inside the Native Backend

3.1.1 Parser and Runtime Representation

The parser in `libclingo/src/heuristic_parser.cc` turns each `__heuristic` symbol into a `HeuristicRuleTemplate`: target atom, positive and negative body literals, variable bindings, modifier, semantics, and an arithmetic AST for the weight.

During `init`, the propagator inspects the ground symbolic atoms and builds an index mapping each predicate and its argument tuple to the corresponding solver literal. It then materializes candidates: for each ground target atom and matching combination of positive body atoms, it creates a `RuntimeHeuristicCandidate` storing the target literal, target tuple, body literals, bound variable values, and instantiated aggregate keys.

In BSP with $n = 100$, this yields only 100 candidates for `b(X)` (and 200 across both partitions), whereas the standard ground encoding generates 1,010,200 directives ($2 \times 100 \times 5051$) to cover all possible discrete sums $S \in \{0, \dots, 5050\}$. The target atoms themselves are still grounded by *gringo* as usual: what the propagator avoids is not the grounding of heuristics per se, but the enumeration of the heuristic directive over the domain of the aggregate. Candidate materialization therefore scales linearly with the number of ground targets rather than with the domain size of dynamic aggregates, and the aggregate value is computed at search time instead of being fixed by a ground directive. This is a narrower form of laziness than *Alpha*, which instantiates the whole program on demand.

Finally, the propagator registers watches exclusively for literals that affect heuristic state: both polarities of target and negative body literals, positive body literals, and aggregate source literals.

3.1.2 Incremental Aggregates

Aggregate states implement three methods: `add`, `remove`, and `result`.

Sums and counts (`SumState`, `CountState`) store a single `int`, because adding and

removing values can be done with simple addition and subtraction. In contrast, min and max (`MinState`, `MaxState`) cannot be undone with arithmetic; when the current minimum or maximum is removed during backtracking, the state must know what the next best value was. They therefore use an `std::map<int, int>` that maps each active value to how many times it occurs.

Each source atom updates the `AggregateState` matched by its `RuntimeAggregateKey`: values are added in `propagate` and removed in `undo`.

Under *clingo* semantics, an aggregate is evaluated only after all its source literals are assigned, gating dependent candidates until then. Under *Alpha* semantics, candidates directly read whatever partial aggregate value is currently available on the trail.

3.1.3 Evaluation and Decision

A candidate is *active* when its target literal remains unassigned, all its positive body literals evaluate to true, and every negative body literal is satisfied under the template’s semantics. The two supported semantics differ only in how they treat negative literals:

```

1 static bool negative_literal_is_satisfied(HeuristicSemantics s,
2   ↪   clingo::TruthValue v) {
3     return s == HeuristicSemantics::clingo ? v == clingo::TruthValue::False
4       : v != clingo::TruthValue::True;
5 }

```

Once a candidate satisfies its body conditions, the propagator computes its numerical weight by evaluating its weight AST over the current aggregate values and term arguments. The resulting effect is then applied to the target literal’s `TargetHeuristicState`.

The native propagator adopts four of *clingo*’s domain heuristic modifiers:

- **level**: sets the branching priority to w (higher values are chosen first), leaving the truth polarity unchanged;
- **sign**: sets the preferred truth polarity based on the arithmetic sign of w (positive for *true*, negative for *false*), using $|w|$ to resolve precedence among conflicting sign directives;
- **true**: sets the branching priority to w and forces the truth polarity to *true*;
- **false**: sets the branching priority to w and forces the truth polarity to *false*.

clingo’s remaining modifiers, `init` and `factor`, are not supported because custom propagators cannot manipulate clasp’s internal VSIDS score tables.

If multiple heuristic rules target the same atom, their effects are combined into a `TargetHeuristicState` that tracks the `level` and `sign` across all active `RuntimeHeuristicCandidate` instances.

Unassigned targets with an active `level` are stored in an ordered set:

```

1 std::set<DecisionRankKey, DecisionRankGreater> active_decision_ranks_;

```

Entries are sorted by decreasing weight. Because weights are stored as standard 32-bit integers, this avoids the 16-bit saturation limit of clasp’s internal domain table (Section 2.1.4).

When clasp calls `decide`, the propagator inspects `active_decision_ranks_` from the beginning, discarding any targets that are already assigned on the trail or whose weights have changed. It then branches on the first valid target using its stored `sign`. If the set is empty, the propagator checks whether clasp’s fallback atom has a stored `sign`; otherwise, it returns 0 to delegate the decision to clasp. During backtracking, `undo` restores aggregates and candidate states using `noexcept` functions to prevent state corruption during solver unwinding.

3.2 The Prolog Backend

While the native backend prioritizes evaluation speed through compiled C++ structures, the Prolog backend evaluates heuristics as arbitrary logic programs via an embedded engine, trading runtime overhead for unrestricted expressiveness. This section describes its runtime architecture and trail synchronization mechanisms (with a comprehensive performance comparison presented in Section 4.13).

3.2.1 Rule Strings and Normalization

Section 2.3.2 introduced the notation and the predicates it relies on; recalling the BSP heuristic:

```

1 heuristic("heuristic(b(X), W, 0, true) :-
2   aggregate_all(sum(Y), holds(c(Y)), S), holds(heuristic_base(Base)),
3   holds(x(X)), target_available(b(X)), alpha_not(c(X)),
4   W is Base*S+X. ").

```

During initialization, the propagator collects all `heuristic/1` facts and normalizes the rule strings. The body uses either `clingo_not/1` or `alpha_not/1`, and that choice is what fixes the semantics of the rule. The propagator extracts the predicate signatures referenced in the rules (`holds`, `alpha_not`, `clingo_not`, `target_available`, and rule targets) and tracks only the solver atoms matching those signatures, so that irrelevant predicates cost neither memory nor bookkeeping.

3.2.2 The Runtime Program

During initialization, the backend starts an embedded SWI-Prolog engine [13] and loads a set of base Prolog rules:

```

1 holds(A) :- true_atom(A).
2 holds(A) :- static_atom(A), \+ true_atom(A).
3 clingo_not(A) :- false_atom(A).
4 alpha_not(A) :- \+ holds(A).
5 target_available(A) :- target_atom(A), \+ true_atom(A), \+ false_atom(A).

```

```

6 dyn_max(Goal, Template, Max) :-
7   (aggregate_all(max(Template), Goal, R) -> Max = R ; Max = 0).

```

Most predicates in these base rules are standard static Prolog definitions, asserted once during initialization and remaining invariant throughout search. The key exceptions are `true_atom/1` and `false_atom/1`, which dynamically track the solver’s partial assignment. Rather than residing in the Prolog database as dynamic facts, both are implemented as C++ *foreign predicates* via SWI-Prolog’s C API. They directly query an in-memory literal store maintained by the propagator: two hash sets (one for true and one for false literals), partitioned by predicate signature. Consequently, recording trail changes during `propagate` or `undo` simply updates these native C++ structures, without requiring any calls into the Prolog engine.

Unlike the native propagator, which defers an aggregate until every one of its source literals is assigned (Section 3.1.3), the Prolog engine has no mechanism to suspend `aggregate_all`: the wait has to be expressed in the encoding itself, using `target_available/1`. Since it succeeds on atoms that are still undecided, `\+ target_available(Pattern)` succeeds only when no atom matching `Pattern` is left undecided. The `lc` encodings carry one such guard per dynamic source of each aggregate; in BSP, where the sum ranges over the `c` atoms, the guard is `\+ target_available(c(_))`.

3.2.3 Synchronization and Decision

Trail synchronization is incremental: during search, `propagate` and `undo` record atoms as true, false or unassigned in the store described above, as their solver literals change on the trail.

At each decision point (`decide`), the backend runs the query `heuristic(T, W, P, M)` and collects every solution. A solution is only a proposal: before it can become a decision, its target term must be matched to a solver literal that is still unassigned. This matching cannot be delegated to the rule, because an unrestricted Prolog rule may build a target term that does not occur in the ground program at all, and such a term has no literal for clasp to branch on. Among the solutions that survive, the backend branches on the one with the highest priority and weight. Unlike the native C++ backend, which incrementally maintains a sorted set of active targets, the Prolog backend re-evaluates the heuristic query from scratch at every decision step. This design prioritizes relational expressiveness over per-decision efficiency (evaluated in Section 4.9).

3.3 Benchmark Problems and Encodings

3.3.1 Balanced Sum Problem (BSP)

The BSP divides the set of integers $\{1, \dots, n\}$ into two disjoint sets, `b/1` and `c/1`, such that their sums differ by at most one. It is the motivating example of [9], and the encoding used here as the Alpha reference is the authors’ own `Evaluation/Motivating_Example/Balanced_Sum_Alpha.asp`, taken unmodified

from the supplementary material of that paper (Section 3.3.4). The heuristic greedily assigns unassigned integers to set b with a priority proportional to the current sum of set c , driving the solver toward balanced partitions. Because all decisions share a single priority level, the current sum S acts as the primary ranking term, with the element value X resolving ties between equal sums.

The implementation of the three exploratory variants introduced in Section 2.1.2 on BSP required some specific adjustments:

- **ga_weak**: The ground heuristic directives omit the negative body guards (`not c(X) / not b(X)`), retaining only the static aggregate binding $S = \#sum\{Y : c(Y)\}$ without unrolling over candidate values:

```
1 #heuristic b(X) : x(X), S = #sum{Y : c(Y)}, W = ((n+1)*S)+X. [W, true]
2 #heuristic c(X) : x(X), S = #sum{Y : b(Y)}, W = ((n+1)*S)+X. [W, true]
```

Dropping the `sum_value(S)` domain predicate does not reduce the grounding: the equality binding still forces gringo to enumerate every reachable sum, so **ga_weak** materializes the same $\Theta(n^3)$ directives as **gc** and **ga**. What changes is when they fire, since the equality holds only once every atom of the sum is assigned (Section 4.10).

- **la_aux**: Heuristic conditions and aggregate values are pre-materialized:

```
1 h_b(X,S) :- x(X), not c(X), S = #sum{Y : c(Y)}.
2 h_c(X,S) :- x(X), not b(X), S = #sum{Y : b(Y)}.
```

The lazy propagator language is then instructed to monitor these auxiliary atoms via the `__body` constructor:

```
1 __heuristic(__target(b), __body(h_b, __match(0,0), __bind_arg(s,1)),
2             __weight(__add(__mul(s, B), self)),
3             __semantics(alpha), __modifier(true)) :- B = n + 1.
```

The propagator will track truth-value assignments over the ground `h_b(X,S)` atoms on the trail, testing whether auxiliary trail synchronization reintroduces solver overhead.

- **la_co**: uses the same lazy heuristic as **la**, but instead of computing and comparing the two sums separately:

```
1 sumB(Sum) :- Sum = #sum{Y : b(Y)}.
2 sumC(Sum) :- Sum = #sum{Y : c(Y)}.
3 :- sumB(SB), sumC(SC), (SB - SC)**2 > 1.
```

it enforces the partition balance directly with a single aggregate rule:

```
1 #const min_val = tot / 2.
2 #const max_val = (tot + 1) / 2.
3 :- not min_val <= #sum{ Y : b(Y) } <= max_val.
```

This removes the heavy grounding bottleneck of the base problem while keeping the meaning of the heuristic unchanged.

3.3.2 Partner Units Problem (PUP)

The PUP [14] assigns communication units to interconnected zones and sensors under capacity and topology constraints. The heuristic uses a two-level priority policy: zones are assigned first, followed by sensors.

The base encoding is taken from the Alpha reference literature [14, 9]: the reference file is the authors’ `Evaluation/PUP/Double/encoding-alpha-prologheu-double.asp`, and the `double-N` instances are byte-identical to theirs, so both systems run on the same files. However, because the original encoding was written for a lazy solver, running it directly in clingo causes a severe grounding bottleneck. To mitigate this while preserving exact solution equivalence, the encoding was refactored using standard ASP modeling practices: bounding unit indices to valid ranges, expressing capacity limits via linear `#count` aggregates, and eliminating a redundant blocking rule that generated excessive ground constraints.

Finally, the global priority of zones over sensors is flattened into the heuristic weights using the +100,000 offset introduced in Section 2.1.3.

3.3.3 House Reconfiguration Problem (HRP)

The HRP [15] places items into cabinets and assigns cabinets to rooms under person-ownership and dimensional constraints, minimizing deviations from an existing legacy setup. The heuristic enforces a five-level priority policy: reusing legacy components, placing long items, placing short items, assigning cabinets to rooms, and finally setting unassigned choice atoms to false. It is the heuristic of Case Study 1 of [8]; the reference file is the authors’ `DomainHeuristics/House/house_alpha_2021-02-04.asp`, and the encodings of this suite are a rule-by-rule port of it.

Unlike BSP and PUP, the HRP heuristic uses no dynamic aggregates: its conditions are ordinary literals and its weights plain arithmetic, and the aggregates of the encoding sit one rule further away, in the definitions of `fullCabinet/1` and `fullRoom/1`. HRP therefore isolates the behavior of lazy default negation (`!a vs. !c`) on partial assignments.

3.3.4 Alpha as an External Reference

We benchmark our propagator against the lazy-grounding solver Alpha [7, 8] as an external state-of-the-art reference. Specifically, we test the *Qh* (Query Heuristics) variant developed by Comploi-Taupe et al. [9], invoked with `-uqh`, which evaluates domain heuristics and dynamic aggregates over partial assignments via Prolog queries. This mechanism directly mirrors our Prolog backend and implements the semantics our native propagator reproduces.

The choice of build matters: the official Alpha 0.7.0 release does not support `#heuristic` directives, and the main upstream branch evaluates aggregates statically. We therefore use the authors’ research *Qh* build, which implements dynamic

aggregates over partial assignments.¹ All Alpha encodings and benchmark instances are taken unmodified from that same repository: BSP and PUP from its **Evaluation** directory, the HRP encoding from the test resources of [8] it also carries.

We evaluate Alpha under two configurations on all benchmark families: **alpha_qh** (with domain heuristics enabled via **-uqh**) and **alpha_noheur** (the baseline solver without heuristics, **-ids**). On HRP, we also run **alpha_dom**, which evaluates domain heuristics natively without the Prolog bridge. In Section 4.7, we compare these runs against our native clingo variants, **la** (lazy propagator), **ga** (ground unrolled simulation), and **gc_noheur** (heuristic-free baseline), to isolate the cost of lazy grounding from the benefit of dynamic heuristic guidance.

Search counters cannot be compared across systems: Alpha’s *guesses* and *backtracks* measure lazy CDNL search interleaved with dynamic rule instantiation, whereas clasp records decision *choices* and *conflicts* over a pre-ground program. Cross-system comparisons are therefore limited to external process metrics recorded by **runlim**: wall-clock time and peak resident set size (RSS). Furthermore, because Alpha runs on the Java Virtual Machine (JVM), its peak RSS includes the heap pre-allocated via **-Xmx**, providing an upper bound on operational memory rather than the state held by the algorithm itself.

3.4 Benchmark Infrastructure and Metrics

The experimental campaigns are orchestrated using **benchmark-tool** [16], the Potassco benchmarking suite designed to automate ASP experiment execution and distribute jobs across SLURM clusters. Each run executes a specific (system, setting, instance) configuration under the supervision of **runlim** [17], which samples resource consumption while enforcing hard wall-clock time and memory limits.

All clingo runs are executed with domain heuristics enabled (**-heuristic=Domain**) and search configured to find a single answer set (**-n 1**). For each run, we record total execution time, solving and grounding components, peak resident memory (RSS), and search statistics including choices and conflicts. We also track the size of the ground heuristic component, comparing materialized directives against lazy facts, along with propagator-specific overheads such as decision calls, query evaluation times, and trail synchronization costs.

Memory is measured as the peak resident set size (RSS) of the entire solver process, which for the Prolog backend includes the embedded SWI-Prolog runtime. Because the engine incurs a constant initialization overhead, lazy Prolog runs can consume more memory than ground variants on small instances; the advantage over the ground-and-solve variants appears as the instances grow.

Ground variants count directives materialized by gringo, whereas lazy variants count template facts that expand into candidates dynamically during search. The runtime effort deferred by lazy grounding is thus captured directly through the per-decision propagator metrics.

¹https://github.com/tilmanni/Alpha/tree/domspec_heuristics_extended_prolog, the build referenced by [9].

Chapter 4

Results

4.1 Experimental Setup and Dataset

All experiments were executed on dedicated HPC cluster nodes requesting 4 cores, with hard limits of 600 seconds and 30 GB of RAM per run enforced by `runlim`. Both backends were compiled on the test hardware and the version of SWI-Prolog used was 10.0.2.

The evaluation covers the benchmark suite defined in Section 3.3:

- BSP: $n \in \{10, 20, \dots, 200\}$ (20 instances);
- PUP: `double- $N$` , with $N \in \{20, 40, \dots, 200\}$ (10 instances);
- HRP: `house- $N$` , with $N \in \{2, 4, \dots, 20\}$ persons (10 instances).

Across all benchmark families, every variant fails first by timeout (denoted $T@N$ in the summary tables): the 600-second limit is always reached before the 30 GB ceiling, so the frontiers reported here are time-bound throughout. Memory exhaustion does occur, but only on BSP at $n = 180$, where grounding the base program alone exceeds the budget; that is beyond the first failure point of every BSP variant and therefore never determines one.

On BSP, search trajectories between C++ and Prolog coincide identically across all solved instances. On PUP and HRP there are minor discrepancies in choices that arise solely from the tie-breaking among equal-rank candidates.

4.1.1 Campaign Overview and Failure Modes

Tables 4.1–4.3 summarize the outcome of the benchmark campaign, reporting the number of solved instances and the first failure point for each variant. In the BSP benchmark, solver failures scale predictably with instance size: once a variant fails at a given size, it also fails for all subsequent sizes, with timeouts representing the first failure mode in all cases (between $n = 70$ and $n = 170$). On PUP, however, difficulty does not scale monotonically with size: `lc` times out on `double-80` yet successfully solves `double-100`. This behavior stems from the benchmark design, which provides only a single instance per size with significant variations in hardness between neighboring sizes; Section 4.5 analyzes this effect in detail using choice counts rather than runtimes.

Variant	native		Prolog	
	solved	first failure	solved	first failure
gc_noheur	15/20	T@160	15/20	T@160
gc	13/20	T@140	13/20	T@140
ga	6/20	T@70	6/20	T@70
ga_weak	14/20	T@150	14/20	T@150
la	16/20	T@170	15/20	T@160
lc	15/20	T@160	15/20	T@160
la_aux	6/20	T@70	8/20	T@90
gc_noheur_co	20/20	—	20/20	—
gc_co	20/20	—	20/20	—
ga_co	20/20	—	20/20	—
ga_weak_co	20/20	—	20/20	—
la_co	20/20	—	20/20	—
lc_co	20/20	—	20/20	—

Table 4.1: BSP campaign overview ($n = 10..200$, 600 s / 30 GB per run). The top block holds the four main variants, the heuristic-free baseline and the **ga_weak** ablation of Section 4.10; the middle block holds the **la_aux** ablation of Section 4.11; the bottom block is the **_co** family of (Section 4.12).

Variant	native		Prolog	
	solved	first failure	solved	first failure
gc_noheur	4/10	T@100	4/10	T@100
gc	3/10	T@80	3/10	T@80
ga	10/10	—	10/10	—
la	10/10	—	10/10	—
lc	4/10	T@80 (non-mon.)	2/10	T@60

Table 4.2: PUP campaign overview (double- N , $N = 20..200$, 600 s / 30 GB per run). double-200 is the largest instance published with the reference encoding; dashes indicate “solved every available instance”

Variant	native		Prolog	
	solved	first failure	solved	first failure
gc_noheur	10/10	—	10/10	—
gc	10/10	—	10/10	—
ga	10/10	—	10/10	—
la	10/10	—	10/10	—
lc	8/10	T@18	6/10	T@14

Table 4.3: HRP campaign overview (house- N , $N = 2..20$ persons, 600 s / 30 GB per run).

The maximum instance size solved provides different insights depending on the problem of choice.

- **On BSP**, the solvability frontier alone does not separate the approaches. Variants sharing the same base encoding (`gc_noheur`, `1a`, and `1c`) all stop between $n = 150$ and $n = 160$, constrained by the heavy grounding of the base program. What distinguishes them is search efficiency: `1a` reduces solving time and conflicts to near zero (Sections 4.3 and 4.9).
- **On PUP** (Table 4.2), the Alpha-like dynamic guidance lets both `ga` and `1a` solve the entire range up to $N = 200$, whereas the other variants stall around $N = 80$ under search space explosion. With search no longer the bottleneck, PUP isolates the cost of the heuristic representation itself: `1a` halves both grounding time and peak memory with respect to the static aggregate unrolling of `ga` (**12.7 GB** vs **24.4 GB** at $N = 200$).
- **On HRP**, the frontier is uninformative because nearly all variants solve the entire suite; the evaluation centers instead on decision quality and on the misguidance observed in `1c`.

Three further trends are visible in the tables. First, the static ground simulation (`ga`) fails on BSP already at $n = 70$, nine sizes earlier than the baseline, under the combinatorial explosion of static unrolling. Second, the native C++ and Prolog backends yield identical frontiers whenever propagator queries remain cheap; where they diverge, the Prolog backend always fails earlier due to evaluation overhead, particularly for `1c`, where the propagator is queried tens of thousands of times. Third, `1c` shows two distinct failure modes of clingo-style aggregate evaluation over partial assignments: on BSP the heuristic never activates, because its aggregate conditions are not satisfied at any decision point, whereas on HRP delayed evaluation produces misleading suggestions that degrade search by orders of magnitude (Section 4.8).

ANALYSIS PART I — WHAT LAZY EVALUATION BUYS

4.2 Reduction of the Ground Heuristic Component

The primary benefit of lazy evaluation is eliminating the materialization of heuristic directives during grounding. Figure 4.1 and Table 4.4 compare the number of heuristic entries the grounder materializes across instance sizes on BSP between the ground variants (`gc`, `ga`, `ga_weak`) and the lazy variants (`1a`, `1c`).

As established in Section 3.3.1, static aggregate unrolling incurs a theoretical $\Theta(n^3)$ combinatorial explosion. The empirical counts reflect this scaling directly: the ground variants scale rapidly from 1,120 directives at $n = 10$ to over 1.34 million at $n = 110$. In contrast, the lazy representation remains invariant at exactly two template facts across all sizes, yielding a reduction factor of over $670,000\times$ at $n = 110$.

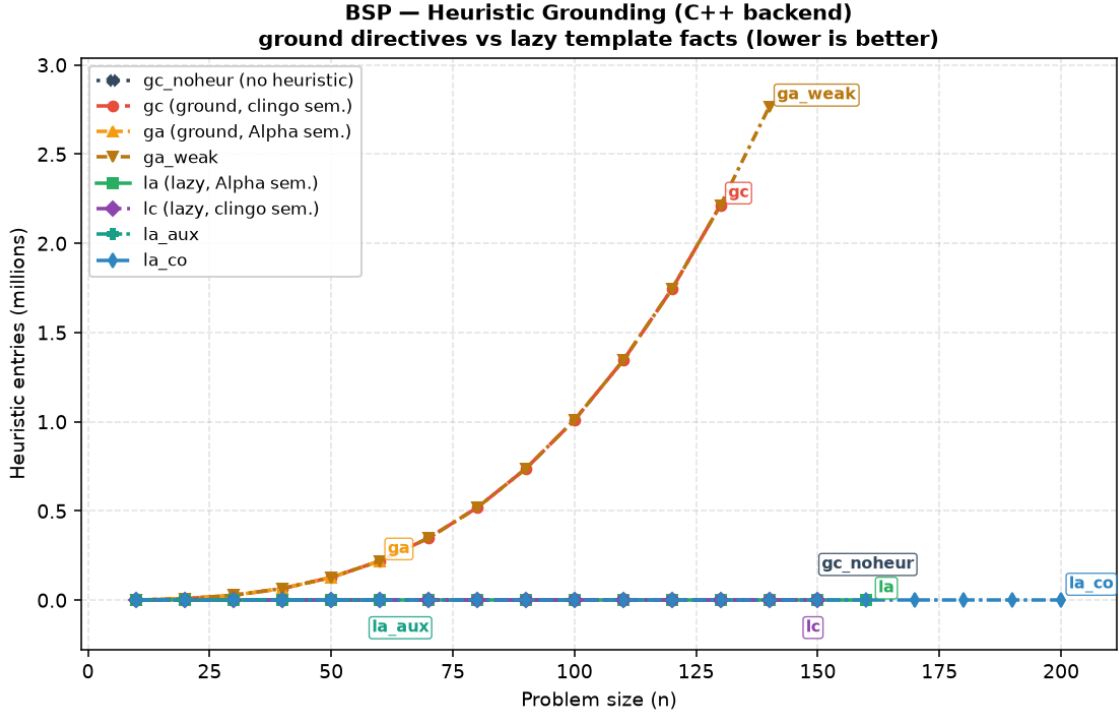


Figure 4.1: Ground heuristic directives against lazy template facts on BSP. The three ground variants coincide by construction, since they unroll the same directive over the same value domain and differ only in the guard that fires it; the difference among the four lazy ones and the heuristic-free baseline are indistinguishable at this scale, at two facts and none respectively. Each curve ends at the largest size its variant solves (Table 4.1), so `ga` stops at $n = 60$ although the count itself keeps growing.

n	Ground directives	Template facts	Ratio
10	1,120	2	560×
50	127,600	2	63,800×
100	1,010,200	2	505,100×
110	1,343,320	2	671,660×

Table 4.4: Growth of the ground heuristic component (`gc`, `ga`, `ga_weak`) against the lazily evaluated one (`la`, `lc`). The two columns count different objects, ground `#heuristic` directives against DSL template facts, so the ratio measures how much representation the lazy form never has to build; it is not a speedup.

Crucially, this reduction specifically eliminates the quadratic unrolling over the dynamic aggregate domain (n^2); ground candidate targets (n) continue to be registered at initialization (Section 3.1.1), with candidate evaluation lazy to search.

4.3 BSP: Runtime Behaviour

For all variants sharing the base encoding without grounded heuristic directives (`gc_noheur`, `1a`, and `1c`), grounding dominates overall execution time. As shown in Figure 4.2, their grounding curves coincide across the entire benchmark range, as none of them materializes heuristic rules during grounding.

The advantage of dynamic evaluation emerges in search performance (Figure 4.3): while `gc_noheur` and `1c` require 44.8s and 49.1s of solving at $n = 120$, `1a` completes the search in just 0.04s and remains near zero across all instance sizes.

Although `1a` achieves the lowest total runtime among the three variants (Figure 4.4), the overall improvement is approximately 25%. Dynamic evaluation under the Alpha-like reading therefore renders search practically negligible, but overall scaling on standard BSP remains governed by the grounding of the base program, a limitation examined further below.

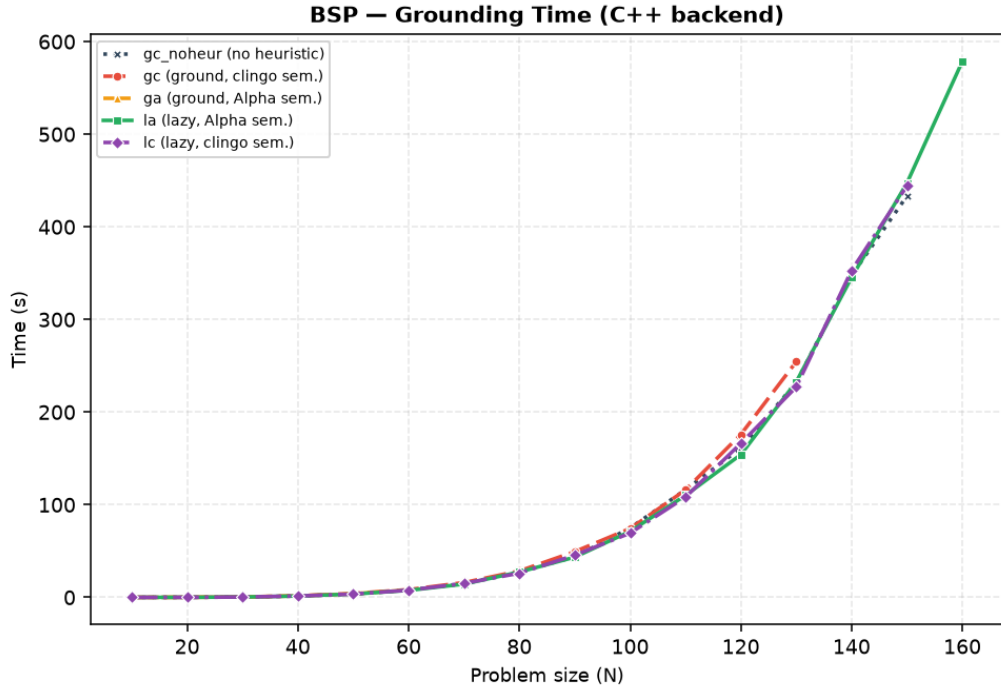


Figure 4.2: Grounding time on BSP for the C++ backend. One run per point; a curve ends where its variant stops solving within the limits.

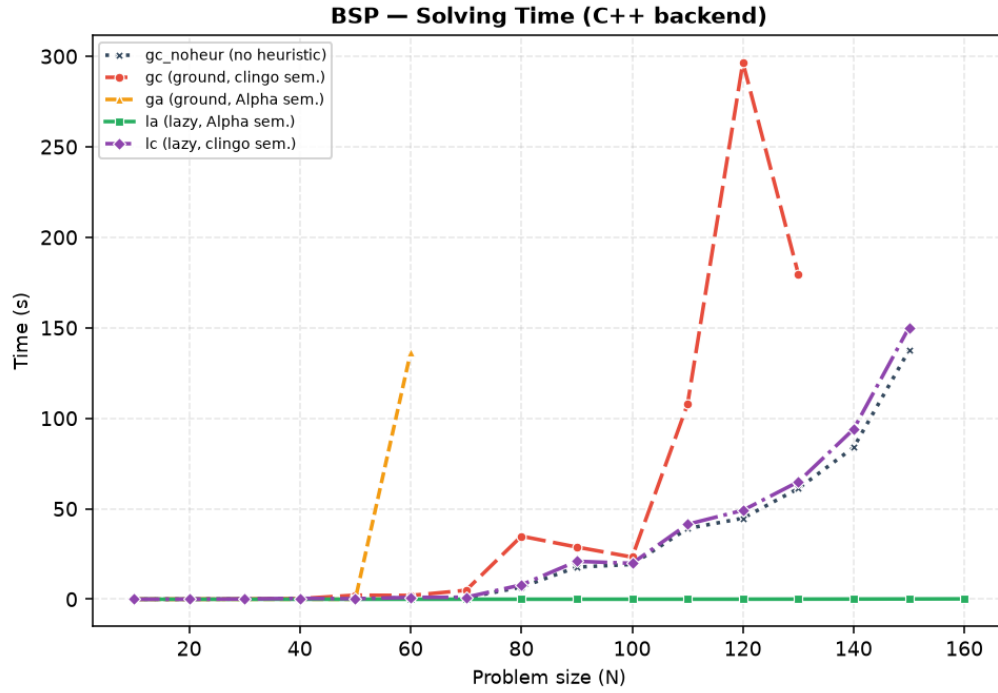


Figure 4.3: Solving time on BSP for the C++ backend, on the same runs as Figure 4.2.

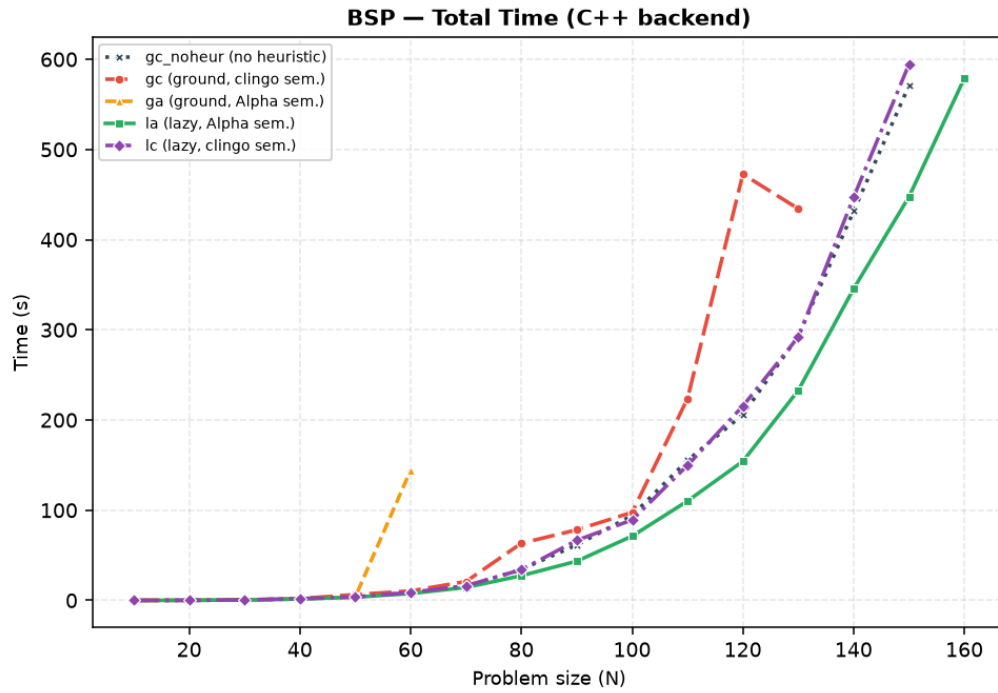


Figure 4.4: BSP total time, grounding plus solving, on the C++ backend.

Variant	Total (s)	Grounding (s)	Solving (s)	Choices	Conflicts
gc_noheur	206.0	161.3	44.8	78,565	49,993
gc	472.6	176.0	296.6	231,260	160,115
ga	T	161.6	> 438.4	783,034	571,602
ga_weak	229.9	163.0	66.9	121,159	80,847
1a	154.2	154.2	0.04	120	0
1c	215.5	166.3	49.1	78,565	49,993
gc_noheur_co	0.016	0.016	< 0.01	37	0
gc_co	6.98	6.83	0.15	65	4
ga_co	4.16	4.08	0.08	74	3
ga_weak_co	3.70	3.63	0.07	74	1
1a_co	0.006	0.006	< 0.01	117	0
1c_co	0.007	0.007	< 0.01	37	0

Table 4.5: Execution time and solver statistics on BSP at $n = 120$. The two clingo-semantics variants are analyzed in Section 4.8; the `_co` block, which shares the heuristics of the block above and differs only in the formulation of the balance constraint, is analyzed in Section 4.12.

The search statistics in Table 4.5 and Figure 4.5 highlight three key observations:

1. In **1a**, default negation and dynamic aggregate evaluation let clingo use the heuristic from the first decision, taking exactly 120 choices with zero conflicts at $n = 120$. Figure 4.5 shows that this is not a property of one size: **1a** requires exactly n decisions with zero conflicts at every benchmarked n . Dynamic aggregate evaluation gives the propagator usable guidance from the first decision on an empty assignment, and every choice it proposes lies on the solution path.
2. **ga** matches **1a** exactly up to $n = 50$, then degrades at $n = 60$ and times out from $n = 70$ onward. The cause is clasp’s 16-bit weight field, as explained in Section 2.1.4. Section 4.10 analyzes the two ground simulations in detail.
3. The frontier itself, however, is not moved by any of this. **gc_noheur**, **1c** and **1a** stop within one size of each other, because around $n = 160$ grounding the base program, already 52.8 million rules at $n = 120$, consumes the budget on its own, before search begins. On BSP the claim is therefore about cost and not about reach; Section 4.5 exhibits the family where it is about reach, and Section 4.12 what happens when the base program is made compact as well, including what becomes of the weight-field limit just described, which turns out to be a limit on the ranking and not on the encoding.

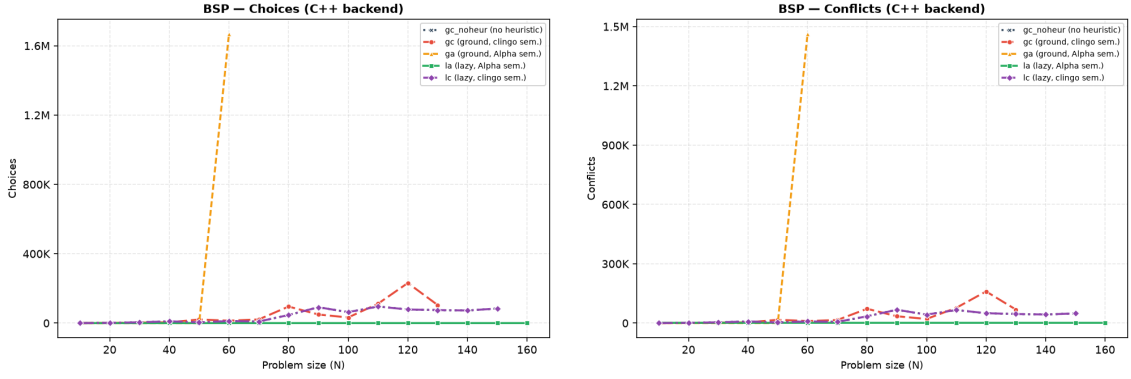


Figure 4.5: BSP search statistics on the C++ backend: solver choices (left) and conflicts (right) across sizes.

4.4 BSP: Rules, Variables, and Memory

The reduction of the heuristic footprint is also evident in the size of the propositional formula generated for the solver. As shown in Table 4.6, at $n = 100$ the two lazy variants replace the 1,010,200 ground directives of `gc` with the two template facts of the DSL, and their variable count (20,300) coincides exactly with the heuristic-free baseline: the heuristic leaves no trace in the propositional formula. The ground variants pay this cost in different forms. In `gc` the directives dominate the formula itself, which grows to more than one million variables, fifty times the baseline; in `ga` the variable count stays close to the baseline, but the same million directives still have to be grounded, and beyond $n = 50$ this effort does not translate into usable guidance. Only `la_co` falls below the baseline, with 101 variables, because it also reformulates the balance constraint (Section 4.12).

Variant	Heuristic entries	Solver Variables
<code>gc</code>	1,010,200	1,030,606
<code>ga</code>	1,010,200	20,402
<code>ga_weak</code>	1,010,200	20,406
<code>lc</code>	2	20,300
<code>la</code>	2	20,300
<code>gc_noheur</code>	0	20,300
<code>la_co</code>	2	101

Table 4.6: Heuristic grounding footprint and propositional solver variables at $n = 100$, C++ backend. The first column counts the heuristic entries gringo materializes: ground `#heuristic` directives for `gc`, `ga` and `ga_weak`, the two DSL template facts for the lazy variants, none for `gc_noheur`.

Figure 4.6 shows that on large instances, memory usage is dominated by the base problem encoding; thus, while lazy heuristics eliminate the memory overhead of heuristic directives, scaling on BSP remains bounded by the ground footprint of the base program, a limitation addressed by constraint reformulation in Section 4.12.

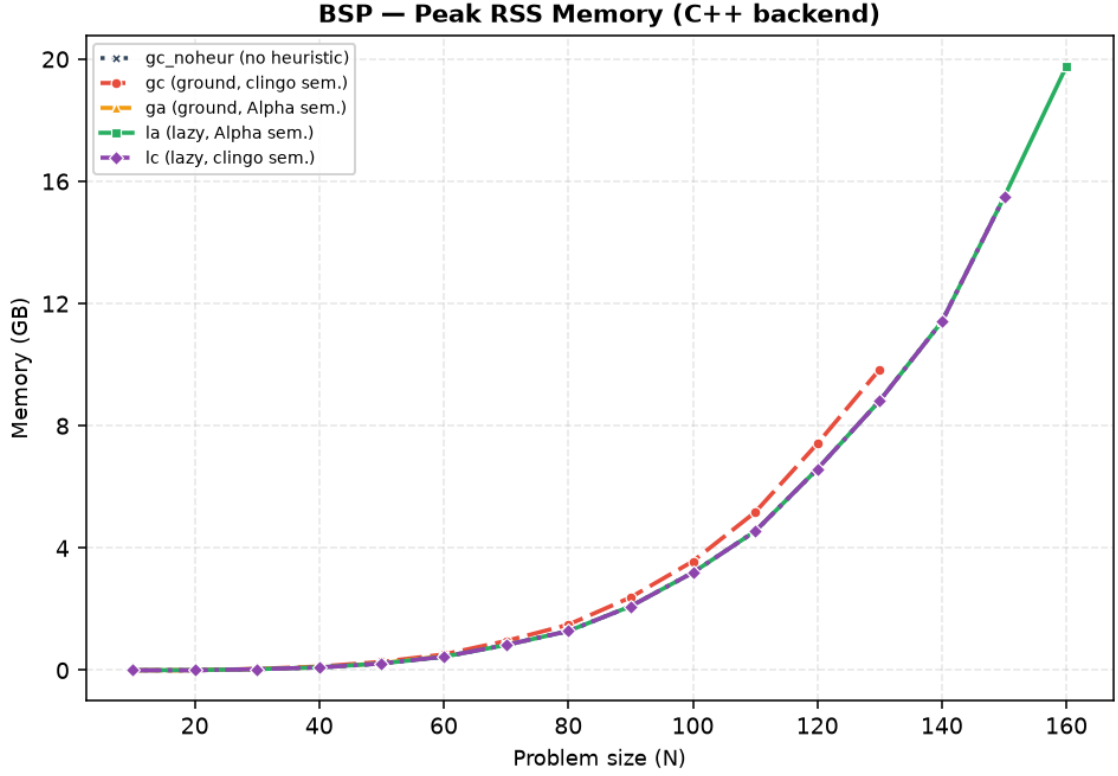


Figure 4.6: BSP: peak resident set size on the C++ backend across instance sizes.

4.5 PUP: the Grounding Bottleneck at Scale

Unlike BSP, which uses only `#sum`, PUP combines dynamic `#count` and `#max` aggregates, the latter tracking the remaining capacity across adjacent partner units.

While in BSP, unguided search dominated baseline execution times, and the base program owns the frontier for every variant that shares it. In PUP, the dynamic Alpha-like semantics reduces solving effort to a few hundred decisions at every instance size, so more than 95% of the total execution time moves to grounding and the cost becomes one of memory. With search removed from the picture, what separates two variants is exactly the size of the heuristic representation they had to build, which is the quantity Section 4.2 measures in the abstract, here measured in seconds and gigabytes.

Figure 4.7 and Table 4.7 summarize the overall performance across all five variants under a 600 s timeout and a 30 GB memory limit.

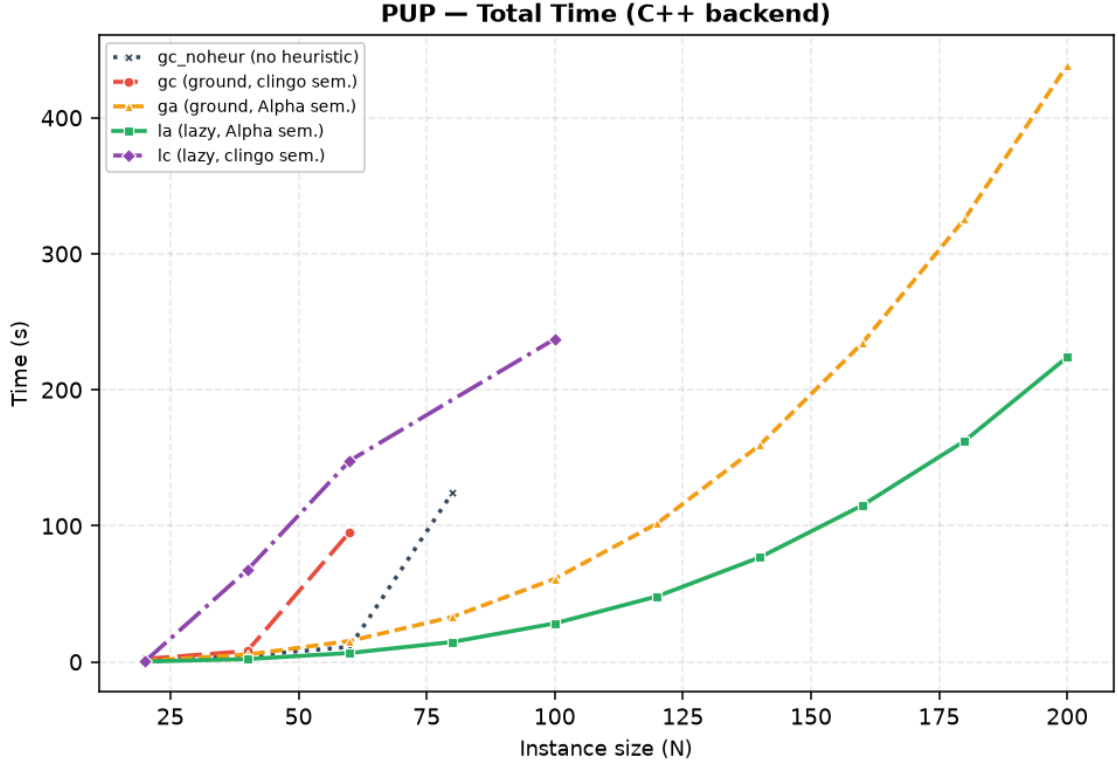


Figure 4.7: Total clingo runtime on PUP across instance sizes for the C++ backend.

Variant	max N solved	Total (s)	Grounding (s)	Peak RSS	First failure
gc_noheur	80	124.3	10.7	0.7 GB	T@100
gc	60	95.5	17.6	1.3 GB	T@80
ga	200	438.6	430.1	24.4 GB	—
la	200	224.0	218.5	12.7 GB	—
lc	100	237.4	27.1	2.1 GB	T@80 (non-mon.)

Table 4.7: PUP frontier on the C++ backend (600 s / 30 GB budget): largest solved double- N instance and corresponding resource metrics. **double-200** is the largest instance of the reference set, so for **ga** and **la** the reported figures are the cost of the last available instance, not the cost of failing.

The unguided baseline. Without dynamic guidance the solver relies entirely on VSIDS, and combinatorial choices escalate rapidly: **gc_noheur** takes 546,549 choices at $N = 80$ and times out at $N = 100$, where it reaches 3.38 million. The two clingo-semantics variants **gc** and **lc** fail in the same region and for reasons that belong to Part II; Section 4.8 analyzes them together with their BSP and HRP counterparts.

Grounding and memory savings (la vs ga). Both Alpha-like semantics variants (**ga** and **la**) solve the entire benchmark horizon up to $N = 200$, maintaining near-perfect search guidance (235 choices for **ga** vs 162 for **la** at $N = 120$). W.r.t **la**, **ga** has dropped the negative guards and reconstructed the **#max** capacity terms through unrolled bounds (Section 2.1.1), and its zone directives, whose flattened

weights start at 100,000, all saturate clasp’s 16-bit field (Section 2.1.4). That last effect does not invert the two priority levels, since the largest sensor weight on these instances is 26,220 and stays under the ceiling, so zones are still decided before sensors; what it costs **ga** is the ranking *within* the zone level, which **la** keeps because it ranks its targets internally. Because solving takes only a few seconds (Figure 4.8), the overall performance is nonetheless differentiated almost exclusively by the grounding footprint.

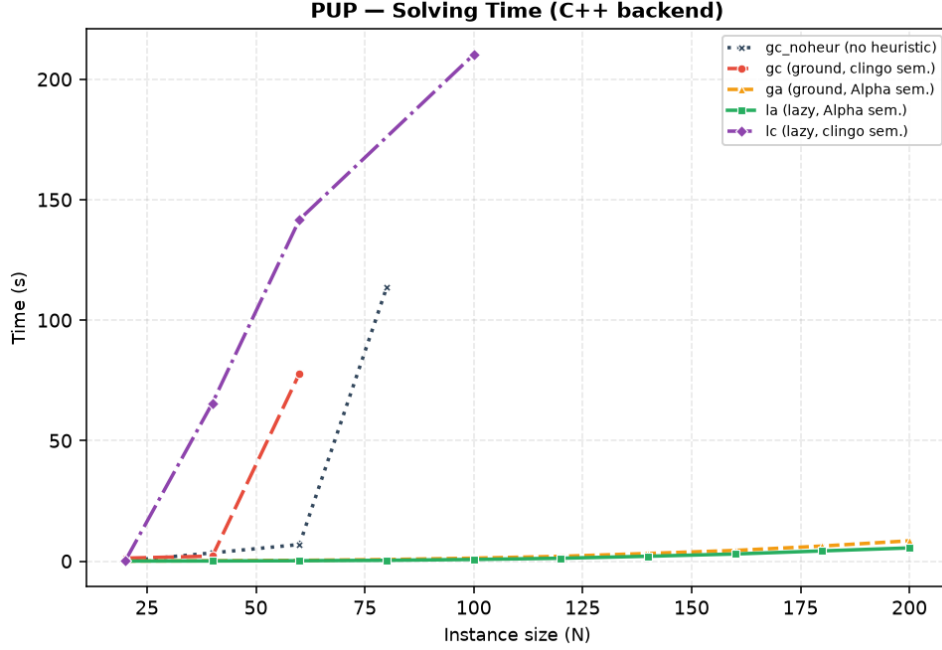


Figure 4.8: Solving time on PUP for the C++ backend. Under the Alpha-like reading **ga** and **la** stay flat along the bottom across the whole horizon, reaching only a few seconds at $N = 200$, which is what leaves their separation entirely to grounding and memory. The three variants without usable guidance climb steeply and then stop.

As shown in Figures 4.9 and 4.10, **ga** must unroll all discrete bounds of the dynamic aggregates into the ground program. At $N = 200$ this costs 430.1s of grounding and 24.4 GB of memory, against a 30 GB limit. Lazy evaluation in **la** leaves the aggregates to the in-memory C++ propagator and grounds only the base problem specification, reducing grounding to 218.5s and peak memory to 12.7 GB. The saving holds across the whole range: the ratio between the two variants is largest on the small instances and settles just below a factor of two at $N = 200$, in both time and memory.

Why the two do not separate on reach, and what that leaves. **ga** and **la** both report 10/10 in Table 4.2, and it would be wrong to read that row as the absence of a difference between them. The **double** family published with the reference encoding ends at **double-200** [14, 9]; every instance that exists was solved by both, so this benchmark has no size at which one of the two fails and the other

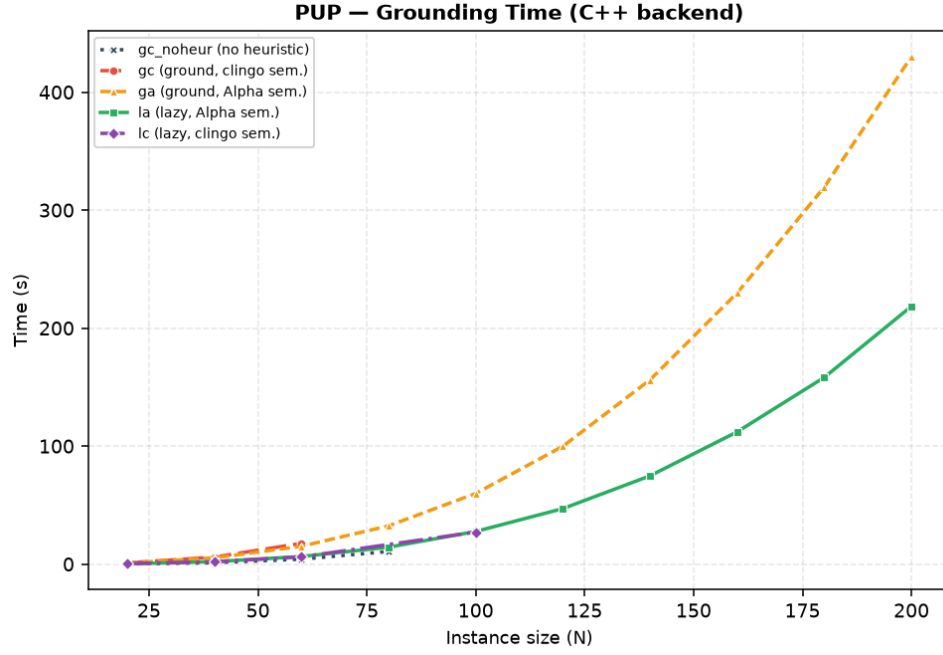


Figure 4.9: Grounding time on PUP for the C++ backend across instance sizes. Note that the vertical scale reaches 430s against the 210s of Figure 4.8, on the same runs.

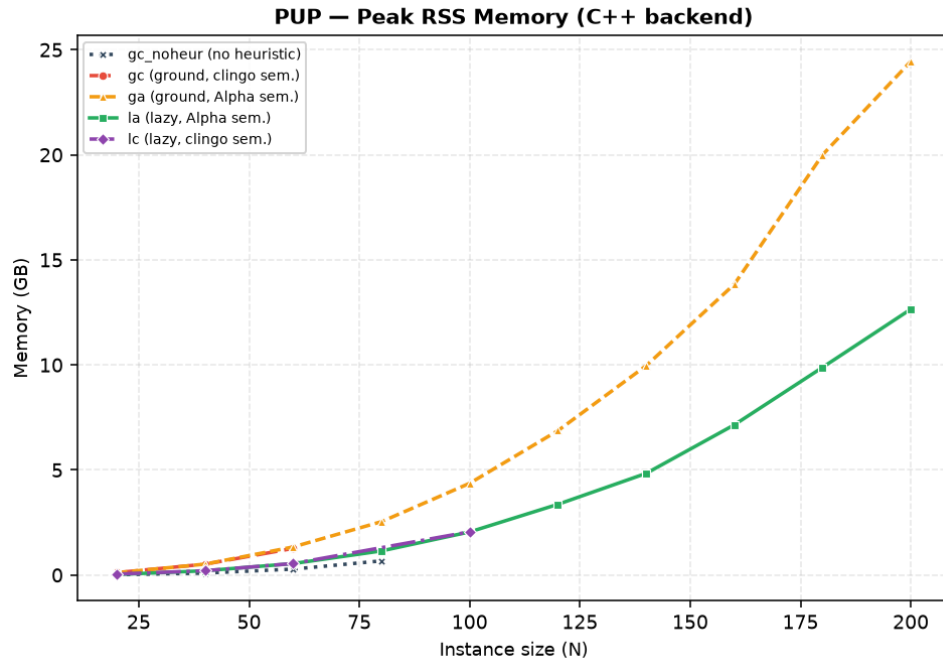


Figure 4.10: Peak memory usage on PUP for the C++ backend, illustrating the 2× memory reduction of la over ga.

does not. The separation it does measure is the one in Table 4.7, and on the last available instance it is not marginal: solving the same problem with the same heuristic costs **ga** 438.6s and 24.4 GB against 224.0s and 12.7 GB for **1a**, a factor of 1.96 in time and 1.93 in memory. The whole of that difference is the heuristic representation, since the two encodings share the base program and their search trajectories agree to within a few hundred decisions.

Where the two frontiers *would* lie can be estimated but not measured. Fitting the two curves over $N \in [160, 200]$ gives peak memory growing as roughly $N^{2.6}$ and grounding time as roughly $N^{2.9}$; extrapolating to the 30 GB ceiling puts **ga** at approximately $N \approx 215$ and **1a** at approximately $N \approx 280$, and the 600s ceiling in the same order. These are extrapolations from four points and are reported as such: they indicate that the measured factor of two in memory is worth about a third in instance size, given how steeply the base program grows, and that confirming it requires instances that the reference material does not contain. Generating them would mean writing a new instance generator and validating it against the ten published files, which is left as future work (Section 5.4); nothing in Part I depends on it, because the claim of this section is a claim about cost at equal reach.

4.6 HRP: Guidance Without Dynamic Aggregates

HRP evaluates heuristic semantics on rules with nested default negations but without dynamic aggregates. It therefore does not test the representation mechanism of Section 4.2 at all: there is no aggregate domain to unroll, so the ground and the lazy forms of the heuristic are comparable in size. What it does test is whether the propagator reproduces a multi-level guidance policy faithfully. Because HRP instances are relatively small, nearly all variants solve every instance in under a second on the C++ backend, so the family measures guidance quality (Figure 4.11) and per-decision overhead (Figure 4.12) rather than reach.

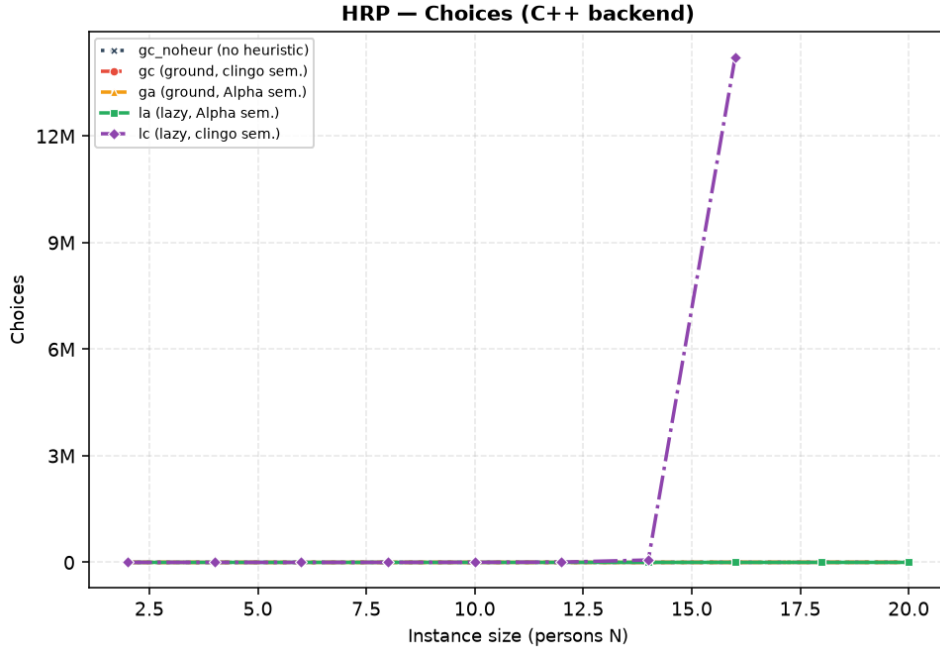


Figure 4.11: HRP: solver choices on the C++ backend. The only diverging curve is `lc`, analyzed in Section 4.8.

Under the Alpha-like reading, `la` and `ga` land within a few choices of each other at every size (86 vs 88 at $N = 14$, 122 vs 116 at $N = 20$, with zero conflicts throughout, Figure 4.11). The agreement says that the lazy propagator and the ground unrolling implement the same ranking policy, which is what the two variants are constructed to share; it is not by itself evidence of fidelity to Alpha. On this family `ga` is not a reference but an approximation, since its ground form drops the capacity guards `not fullCabinet(C)` and `not assignedThing(T)` rather than reinterpreting them (Section 2.1.1); `la` evaluates those guards on the trail. Two variants agreeing does not establish that either matches the reference semantics, and the comparison against Alpha itself is lazy to Section 4.7, which settles the question.

HRP is also the family on which the Prolog backend is most expensive per decision, because the multi-room placement rules join multiple domain predicates and the engine must enumerate over a thousand candidate pairs at every call (Figure 4.12). That cost belongs to the cost model and is quantified in Section 4.9.

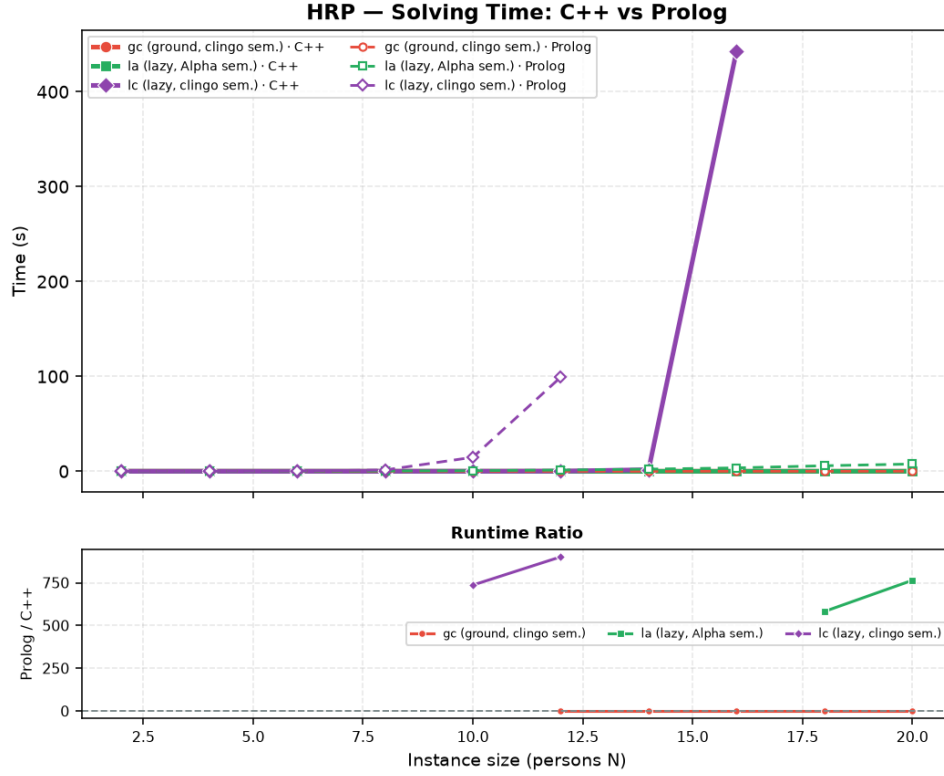


Figure 4.12: HRP: solving time, C++ against Prolog. `lc` is again the only variant whose absolute solving time departs from the others, but the ratio panel below shows that `la` too runs several hundred times slower on the interpreted backend; that gap is a per-decision cost on an otherwise identical trajectory, and Section 4.9 accounts for it.

4.7 An External Reference: Alpha

To evaluate how closely our propagator reproduces the lazy evaluation process and performance advantages, we compare our clingo variants against Alpha with and without its declarative heuristic engine (Qh mode), on all three benchmark families. BSP and PUP carry the dynamic aggregates the mechanism is designed for and are treated first; HRP, whose reference heuristic has none, is covered separately below because it answers a different question.

On BSP, the guided Alpha configuration (`alpha_qh`) solves all twenty instances in 4.2 s and 0.38 GB at $n = 200$, requiring exactly n decisions and zero backtracks. This matches the search trajectory achieved by `la` in clingo (n choices, zero conflicts). The performance gap at larger sizes (579.7 s for `la` at $n = 160$ versus 2.3 s for `alpha_qh`, both on the `runlim` clock) stems entirely from base program grounding in gringo (52.8 million rules at $n = 120$), which Alpha completely avoids by instantiating rules lazily along the search trail.

Lazy grounding alone is not sufficient without dynamic heuristic guidance: as Figure 4.13 shows, `alpha_noheur` solves nothing beyond $n = 40$, eleven sizes short of clingo’s ground baseline `gc_noheur` ($n = 150$). The two systems sit on opposite

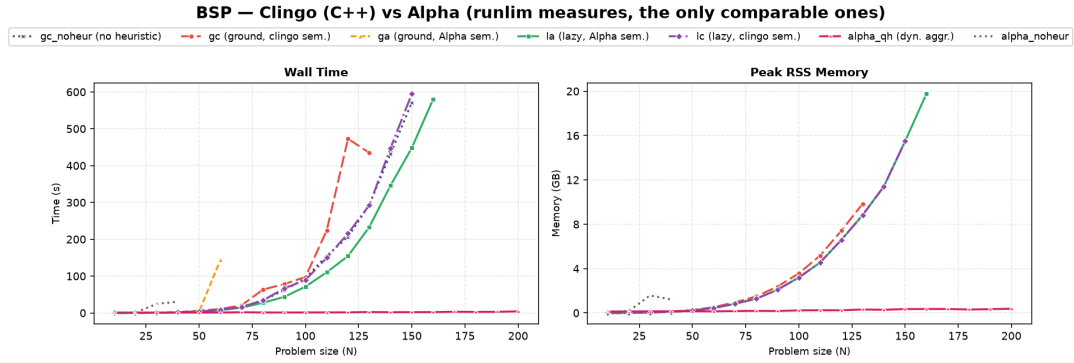


Figure 4.13: BSP benchmark dashboard: the four clingo variants and the heuristic-free baseline `gc_noheur` compared against Alpha with (`alpha_qh`) and without (`alpha_noheur`) declarative heuristics. Left: total wall-clock time; right: peak RSS. While `alpha_qh` stays flat along the horizontal axis, unguided `alpha_noheur` degrades rapidly, failing earlier than any clingo variant.

sides of one trade-off:

- **clingo:** Because the program is fully grounded before search begins, exploring unpromising search branches is computationally very cheap: clasp only has to perform fast boolean propagation and VSIDS variable scoring over pre-existing clauses in compiled C++.
- **Alpha:** Because rules are grounded on demand during search, exploring unpromising branches is extremely expensive: every wrong decision and subsequent backtrack forces the engine to instantiate new rules and nogoods on the fly.

As shown in Figure 4.14, `alpha_noheur` suffers a severe search space explosion, peaking at over 67,000 guesses and backtracks at $n = 30$; it still solves $n = 40$, where each guess costs more even though fewer are needed, and from $n = 50$ it no longer finishes at all. With heuristic guidance, `alpha_qh` never backtracks and materializes only the minimal subset of ground rules along the single valid solution path. The two mechanisms are therefore complementary: dynamic heuristics need lazy evaluation to avoid the grounding bottleneck, and lazy grounding needs dynamic heuristics to avoid paying instantiation cost on every wrong branch.

Partner Units Problem (PUP). On PUP, clingo with lazy heuristics (1a) closely approaches Alpha’s performance, solving $N = 200$ in 227.1s and 12.65GB compared to 50.9s and 12.82GB for `alpha_qh`: a factor of 4.5 in time, and a peak memory that is marginally *below* the reference. The memory figures should not be read as a win, however, and the reason is stated in Section 3.3.4: Alpha runs on the JVM and its RSS reports the heap reserved through `-Xmx` rather than the heap in use, so the honest reading of a ratio of 0.99 is that the two systems are indistinguishable on this axis. In contrast, the static ground simulation `ga` required 444.8s and 24.45GB, nearly nine times the runtime and 1.9 times the memory of `alpha_qh`. Furthermore, `alpha_noheur` solves no instances on PUP, reaffirming that dynamic heuristics are essential for solving this problem family. Figure 4.15 places the three

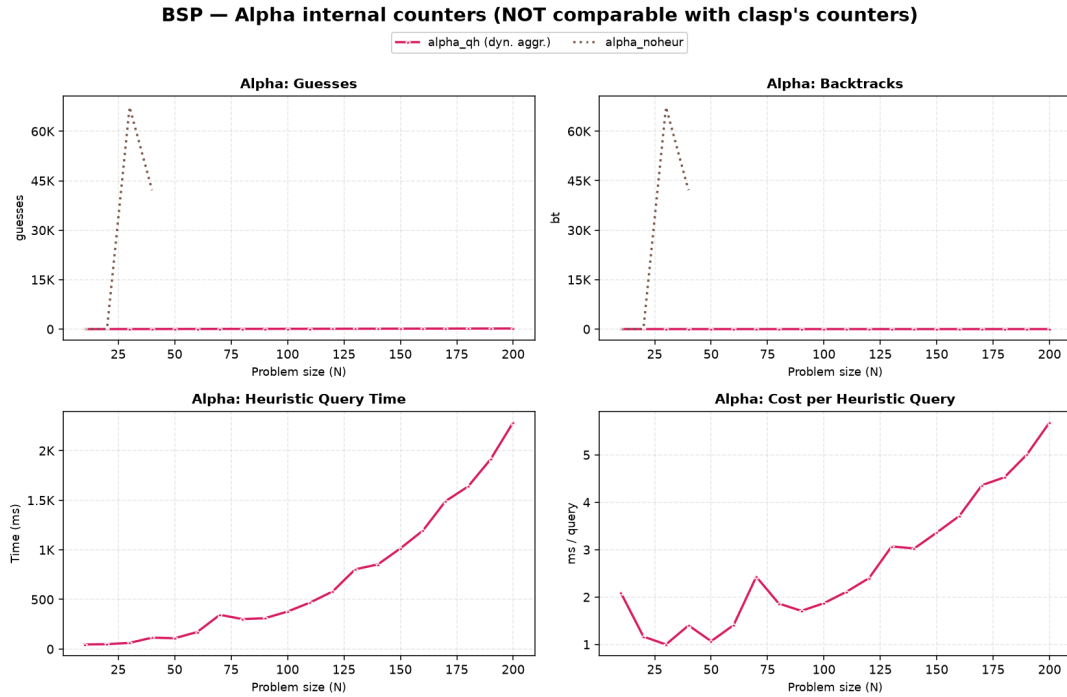


Figure 4.14: BSP internal search counters for Alpha across instance sizes. Top row: with heuristics (`alpha_qh`), guesses remain equal to n with zero backtracks; without heuristics (`alpha_noheur`), search explodes past $n = 20$ and the curve stops at $n = 40$, the last size it solves. Bottom row: cumulative Prolog query evaluation time for `alpha_qh`, which reaches 2.3s at $n = 200$ and never costs more than 5.7 ms per consultation.

systems on the same time and memory axes.

House Reconfiguration Problem (HRP). HRP is the one family whose reference heuristic contains no dynamic aggregates, so it does not test the mechanism this thesis is about; it does, however, settle the question left open in Section 4.6, namely what `1a` should be compared against once `ga` is known to be an approximation. Alpha is run here in all three configurations, including `alpha_dom`, which reads the same domain heuristic natively instead of through the Prolog bridge (Section 3.3.4). Table 4.8 reports the largest instance.

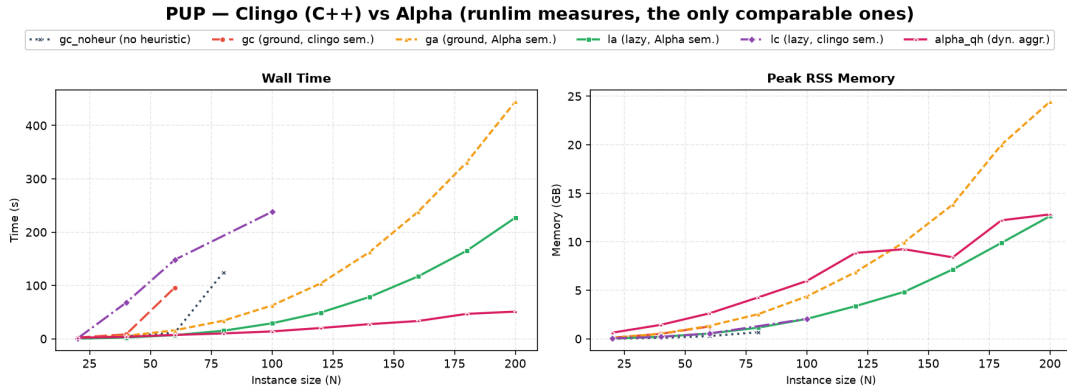


Figure 4.15: PUP benchmark dashboard: external comparison between clingo variants and Alpha. The baseline `alpha_noheur` does not appear as it solves zero instances. The lazy clingo variant `la` remains within a small factor of `alpha_qh` in both time and memory, whereas the ground simulation `ga` incurs substantial overhead on both axes.

Configuration	solved	Wall (s)	Peak RSS	guesses / backtracks
<code>alpha_noheur</code>	3/10	T	—	—
<code>alpha_qh</code>	10/10	15.33	1.70 GB	2,368 / 2,142
<code>alpha_dom</code>	10/10	1.79	0.42 GB	2,314 / 0
<code>clingo la</code>	10/10	0.51	50 MB	122 / 0 (<i>choices / conflicts</i>)

Table 4.8: HRP at `house-20`, the largest instance. Alpha counters are *guesses* and *backtracks* over a lazily instantiated program and are not commensurable with clasp’s *choices* and *conflicts* (Section 3.3.4); they are reported to compare Alpha’s three configurations with one another. `alpha_noheur` solves only `house-2`, `house-6` and `house-8`, the last in 106.0 s with 26,086 guesses.

Three things follow. First, the heuristic matters here as much as on the other two families: without it Alpha solves three instances out of ten and needs 106 s for `house-8`, while both guided configurations clear the suite in seconds. Second, and unlike BSP and PUP, clingo is not the slower system on this family: `la` solves `house-20` in 0.51 s and 50 MB against 15.33 s and 1.70 GB for `alpha_qh`, because the HRP ground program is small and the grounding bottleneck that dominates BSP simply does not arise. HRP is where ground-and-solve with lazy heuristics is on the favourable side of the trade-off of Section 4.9, which is the mirror image of the BSP result and worth stating for that reason.

Third, and least expected, the two guided Alpha configurations do not behave alike: `alpha_dom` reaches the solution with no backtracking at all, whereas `alpha_qh` takes 2,142 backtracks for essentially the same number of guesses, and pays a factor of 8.6 in wall time and 4.1 in memory for them. The two read the same directives; they differ in the evaluation path, native in one case and through Prolog queries over the partial assignment in the other. This is a property of the reference implementation on an aggregate-free heuristic and this thesis does not diagnose it.

Its bearing on Section 4.6 is direct, though: on HRP the backtrack-free trajectory that `la` reproduces is the one `alpha_dom` takes, not the one taken by `alpha_qh`, the configuration whose semantics the propagator was built to mirror. The agreement between `la` and `ga` reported there is therefore consistent with the reference, but it remains an agreement between two implementations of one policy rather than a validation against the system that defines it.

Overall Frontier and Query Engines. Table 4.9 summarizes the maximal instance reach and resource consumption across systems. Regarding query engines, Alpha’s Qh mode and the Prolog backend of this thesis both evaluate heuristics by querying an embedded Prolog engine over partial assignments. Consultation costs are of similar magnitude: at BSP $n = 120$, Alpha issues 242 queries at 2.40 ms each, against 120 consultations at 0.45 ms for our Prolog backend. On this family the embedded-engine design is therefore competitive with the reference engine on consultation cost; Section 4.9 shows that the comparison does not hold uniformly, since the cost per consultation depends on how many candidates the rule body enumerates.

Family	System	max solved	Wall (s)	Peak RSS
BSP	<code>alpha_noheur</code>	40	30.5	1.2 GB
BSP	<code>clingo gc_noheur</code>	150	571.5	15.5 GB
BSP	<code>clingo la</code>	160	579.7	19.8 GB
BSP	<code>alpha_qh</code>	200	4.2	0.38 GB
BSP	<code>clingo la_co</code>	200	0.15	< 0.1 GB
PUP	<code>alpha_noheur</code>	—	—	—
PUP	<code>clingo gc_noheur</code>	80	124.5	0.7 GB
PUP	<code>clingo lc</code>	100	238.2	2.1 GB
PUP	<code>clingo ga</code>	200	444.8	24.4 GB
PUP	<code>clingo la</code>	200	227.1	12.7 GB
PUP	<code>alpha_qh</code>	200	50.9	12.8 GB
HRP	<code>alpha_noheur</code>	8	106.0	2.4 GB
HRP	<code>alpha_qh</code>	20	15.3	1.70 GB
HRP	<code>alpha_dom</code>	20	1.8	0.42 GB
HRP	<code>clingo la</code>	20	0.51	< 0.1 GB

Table 4.9: Largest instance solved within the 600 s / 30 GB budget, and its cost, ordered by reach within each family. On PUP, 200 is the last published instance rather than a frontier for the three systems that reach it. Wall time and peak RSS are `runlim` external process measures; Alpha’s RSS includes the reserved JVM heap. `clingo` rows represent the native C++ backend. A dash marks a configuration that solved no instances.

4.8 The clingo Reading of Partial Information: `gc` and `lc`

The two variants on the clingo side of the semantics axis do not help on any benchmark of this suite, and they fail in ways that are worth separating. This section collects them, because reporting the same negative result three times inside three different family sections obscures both the result and the families.

The outcome should not be read as a defect of clingo’s semantics. The heuristics of this suite were written for Alpha, whose reading of `not a` and of an aggregate over choice atoms is the one the encodings assume; `gc` and `lc` evaluate those same bodies under a semantics that requires the information to be settled before it may be read. What the campaign measures is therefore the cost of the mismatch, and the reason the semantics axis of Section 2.1 has to exist as an axis: it is not a free parameter, and picking the wrong value for a given encoding costs everything the heuristic was supposed to buy. The three families exhibit the two possible outcomes of that mismatch.

4.8.1 BSP: inertness

On BSP, `lc` exhibits search statistics that are identical point-by-point to the heuristic-free baseline `gc_noheur`, recording 78,565 choices and 49,993 conflicts at $n = 120$ (Table 4.5 and Figure 4.5). Under clingo semantics, aggregate guards on `c/1` require all elements to be assigned before the sum can be evaluated, and an atom counts as “assigned false” only once clasp has shown it cannot be true. During search these conditions are never met at a decision point: at $n = 120$ the propagator is consulted on every decision and yields zero applicable candidates across the entire run.

Because `1a` and `lc` run on the same native C++ backend and on a byte-identical ground program, the comparison isolates the semantics exactly. What it establishes is a statement about this encoding, not about the approach: *when* a heuristic body is written so that its guards can only be settled by a complete assignment, moving its evaluation into search buys nothing, because there is no partial information the clingo reading is willing to look at. The BSP heuristic is the extreme case of that shape, since its aggregate ranges over the very choice atoms the heuristic is meant to decide, and its negative guard tests the target’s own complement.

The ground twin `gc` does worse than inert. It carries the same never-firing conditions, but materializes them: the 1,010,200 directives at $n = 100$ enter the propositional formula and multiply the variable count by fifty (Table 4.6). At $n = 120$ this costs 296.6s of solving against the baseline’s 44.8s, and the frontier drops from $n = 150$ to $n = 130$. A heuristic that never fires is not neutral if the solver has to carry its ground image.

4.8.2 PUP: the same inertness, with the same surcharge

PUP repeats the pattern on a different aggregate. `1a` and `lc` produce ground propositional programs that are completely identical (216,193 rules at $N = 20$), differing

exclusively in the semantics selector inside the propagator, so this is again a controlled comparison of the reading alone:

- **1a** evaluates dynamic aggregates over currently true atoms, solving $N = 120$ in 48.2s with only 162 choices.
- **1c** waits for complete assignments, generating zero active candidates during search. At $N = 120$, it makes 49,261 choices and times out.

The unguided nature of **1c** also explains its non-monotonic failure pattern in Table 4.7 (timing out on **double-80** but solving **double-100**). Without heuristic guidance, the solver is subject to the high variance of CDNL restarts, occasionally finding an early solution by chance before failing at larger sizes.

Here too the ground variant is the worse of the two, and for the same reason: **gc** fails at $N = 80$ with 874,565 choices, *earlier* than the heuristic-free baseline, which solves that same size in 546,549 choices and only fails at $N = 100$. The inert ground directives add variables to the solver, enlarge the search space, and degrade VSIDS branching. On both BSP and PUP, therefore, the standard ground heuristic under the standard semantics is measurably worse than writing no heuristic at all.

4.8.3 HRP: misguidance

HRP produces the opposite failure, and it is the more instructive of the two. The HRP heuristic has no dynamic aggregates and is organized in five priority levels; under the clingo reading, the negation in the higher-level rules delays them, while the lower-level rules, whose bodies carry no such guard, fire on incomplete information. The heuristic therefore does *not* go quiet: it actively steers, in the wrong order.

The cost is spectacular. **1c** requires 66,717 choices at $N = 14$ against 86 for **1a**, an increase of nearly eight hundred times; at $N = 16$ it takes 14.2 million choices, five orders of magnitude; from $N = 18$ it no longer finishes (Figure 4.11). Both backends independently reach the same order of magnitude, so this is a property of the guidance and not of an implementation.

On the Prolog backend the same trajectory is the single most expensive configuration in the campaign, and it is the only one the compiled backend solves and the interpreted one does not: misdirected search and relational query cost compound until **house-14**, which the native backend clears in 1.80s, exhausts the whole 600s budget. The Prolog frontier on this family is therefore two sizes short of the native one, $N = 12$ against $N = 16$, and Section 4.9 locates the cost precisely.

4.8.4 What the three cases share

Across the suite, the clingo reading of partial information never produces usable guidance, and the mode of failure depends only on the shape of the heuristic body:

- if every guard needs a complete assignment, the heuristic is *inert* and the run reduces exactly to the unguided baseline (BSP, PUP);
- if some guards need a complete assignment and others do not, the surviving fragment fires out of order and the heuristic becomes *misleading*, which is far worse than inert (HRP);

- in both cases the ground form `gc` is worse than the lazy form `lc`, because an inert directive that has been materialized still enlarges the propositional formula, whereas an inert lazy fact costs two atoms.

Only the second of these was visible before the campaign. The first, and the fact that it degrades gracefully to the baseline instead of degrading the baseline, is what makes the semantics selector a safe thing to expose in the language rather than a trap; the third is the argument for the lazy representation that is independent of the aggregate size.

PART III — WHAT IT COSTS, AND WHAT IT NEEDS

4.9 The Price of Laziness: Per-Decision Cost

The lazy evaluation model represents a space–time trade-off: instead of a combinatorial cost in memory during the grounding phase, there is a cost in computation incurred at each decision made during search. Now that all three families have been presented, that cost can be reported for all of them at once. There is also a remarkable difference between the two backends: the C++ one manages to be orders of magnitude faster in its decisions.

Family	Var.	Decide calls		Cost (ms/decide)		Prolog backend only		
		C++	Prolog	C++	Prolog	cand./call	sync (s)	prop. share
BSP $n=120$	1a	120	120	$0.9 \cdot 10^{-4}$	0.45	121	1.0 ms	0.03%
BSP $n=120$	1c	78,565	78,565	$1.5 \cdot 10^{-4}$	0.07	0.0	0.29 s	2.6%
PUP $N=160$	1a	217	197	$4.4 \cdot 10^{-4}$	9.73	5.3	0.37 s	1.9%
PUP $N=200$	1a	272	249	$5.2 \cdot 10^{-4}$	13.09	5.2	0.63 s	1.7%
HRP $N=20$	1a	122	106	$1.5 \cdot 10^{-4}$	71.93	1,248	3.7 ms	94.5%
HRP $N=14$	1c	66,717	<i>T</i>	$0.9 \cdot 10^{-4}$	—	—	—	—

Table 4.10: Per-decision overhead on representative solved instances of all three families. The last three columns have no C++ counterpart: the Prolog backend re-evaluates the heuristic query from scratch at every decision, whereas the C++ backend keeps its ranked targets incrementally and inspects one entry per call by construction. The `1c` row on HRP has no Prolog figures because that configuration exhausts the time limit and the counters are only emitted on a completed run.

The overall overhead varies dramatically across the benchmark configurations, and Table 4.10 is ordered to make the four regimes visible:

- When the heuristic has **perfect guidance (1a on BSP)**, with only 120 decide calls taking 0.45 ms each in Prolog (and 10^{-4} ms in C++), propagator overhead accounts for less than 0.05% of the total runtime.

- When the semantics gets in the way (**1c on BSP**), each single query is very cheap (0.07 ms) because it finds nothing to return, but the calls are made 78,565 times instead of 120: the propagator ends up costing 2.6% of the total runtime, roughly a hundred times what it costs in **1a**, and none of that work helps the search.
- When candidate generation becomes heavy (**1a on HRP**), producing over 1,200 candidates per decision, Prolog evaluation time rises to 71.9 ms per call; as a result, the propagator absorbs 94% of the total runtime (7.6 s out of 8.1 s, against six milliseconds on the compiled backend), even though search needs only about 100 decisions.
- When frequent search and expensive evaluation collide (**1c on HRP**), the two factors multiply until the run no longer completes: the same 66,717-decision trajectory that the native backend clears in 1.80 s exhausts the 600 s budget on the Prolog one. This is the only configuration in the campaign where the choice of backend costs solved instances rather than seconds.

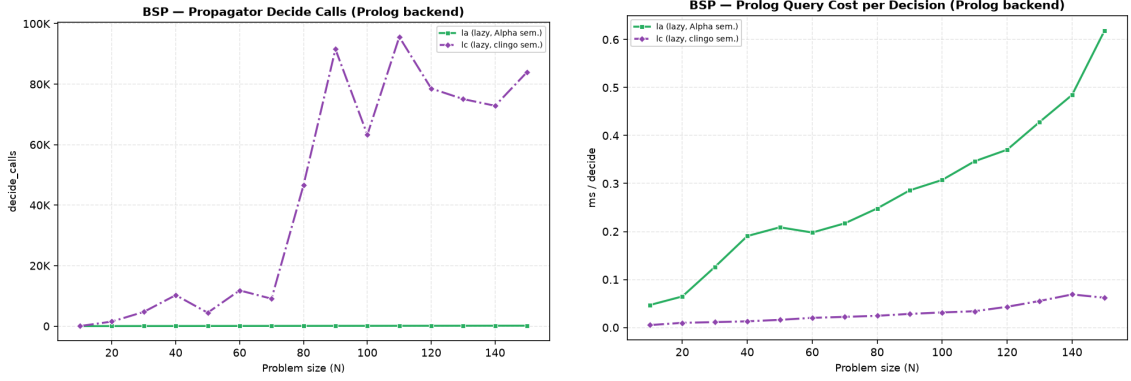


Figure 4.16: BSP, Prolog backend: decide calls and per-decision query cost. The **1c** curve pairs tens of thousands of calls with a low unit cost; **1a** the opposite.

Figure 4.16 separates the two factors on the Prolog backend. Its right-hand plot isolates the inference component alone, the time spent inside the Prolog query, which is the larger part of the per-decision cost that Table 4.10 reports in full: at $n = 120$ it is 0.37 ms of the 0.45 ms for **1a** and 0.043 ms of the 0.067 ms for **1c**, the remainder being candidate matching and literal lookup on the C++ side. Under Alpha semantics, the query successfully identifies and evaluates all eligible candidate atoms at each step, and that is what makes it the more expensive of the two, through Prolog term unification and weight calculation. Under clingo semantics the negative body condition fails immediately on partial assignments, producing zero candidates; while this lets individual queries terminate some eight times faster, it provides no guidance whatsoever to the solver, causing a massive explosion in decision calls.

Where the Prolog cost actually sits. The measurements above are taken on a backend whose synchronization path was rebuilt after an earlier campaign identified it as the bottleneck, and the comparison between the two designs is itself a result. In the original design, every trail change was mirrored into the dynamic Prolog

database by building a goal as a text string and handing it to `PL_chars_to_term`, so each literal assignment paid for Prolog’s reader before touching the database. That cost dominated everything else: on `double-200` it accounted for 17.5s against 1.75s of actual query evaluation. In the current design the trail is not mirrored at all. As described in Section 3.2.3, `true_atom/1` and `false_atom/1` are registered as non-deterministic *foreign* predicates that read the assignment directly from the C++ side, so recording a trail change is a hash-map update with no Prolog call in it.

The effect is visible in Figure 4.17 and in the *sync* column of Table 4.10. On `double-200` synchronization falls from 17.5s to 0.63s, a factor of 27, which brings it level with the 0.68s the native C++ backend spends on the same task: on this axis the interpreted backend is no longer distinguishable from the compiled one. The dominant cost is now inference, not synchronization, with 3.25s spent inside the Prolog query against 0.63s of state maintenance at $N = 200$, a ratio that the previous design had the other way round.

The change is not uniformly favourable, and the place where it is not is instructive. Reading the assignment through a foreign predicate is free when the argument is ground, which is the case for the membership tests inside `alpha_not/1` and `target_available/1`. When the argument is partial, however, the predicate must act as a generator, and it does so by snapshotting the matching relation into a vector before enumerating it. Rule bodies that enumerate heavily therefore pay a cost the dynamic-database design did not, because there the indexing was the engine’s own. HRP is exactly that shape: its placement rules join several domain predicates and produce 1,248 candidates per decision, and its per-call cost rises from 9.19ms under the old design to 71.9ms under the new one, which is what turns `house-20` from a 1.5s run into an 8.1s one and pushes `1c` on `house-14` past the time limit. The trade is a large win wherever synchronization dominated, such as PUP and the `1a_aux` ablation of Section 4.11, which gains five instances, against a loss wherever candidate enumeration does. Section 5.4 states what the remaining half of the fix would be.

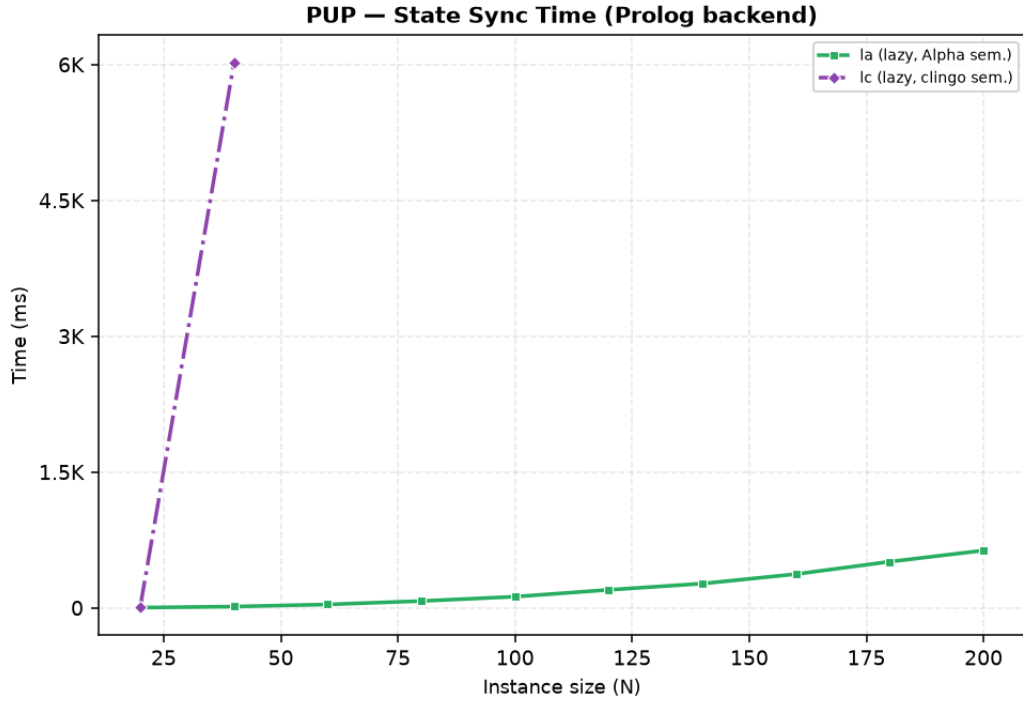


Figure 4.17: Cumulative state synchronization time for the Prolog backend on PUP, with `true_atom/1` and `false_atom/1` as foreign predicates. For `1a` it grows smoothly to 0.63s at $N = 200$, below the 3.25s the same run spends inside the Prolog query; the `1c` curve, which pays one update per literal on an unguided search, reaches 6s already at $N = 40$ and stops there.

As shown in Figure 4.18, solving time for `1a` on PUP stays under ten seconds across the whole horizon on both backends, with the Prolog engine holding a small and stable factor over the native one. Both backends deliver guidance of the same quality here, keeping the search down to a few hundred choices on either engine.

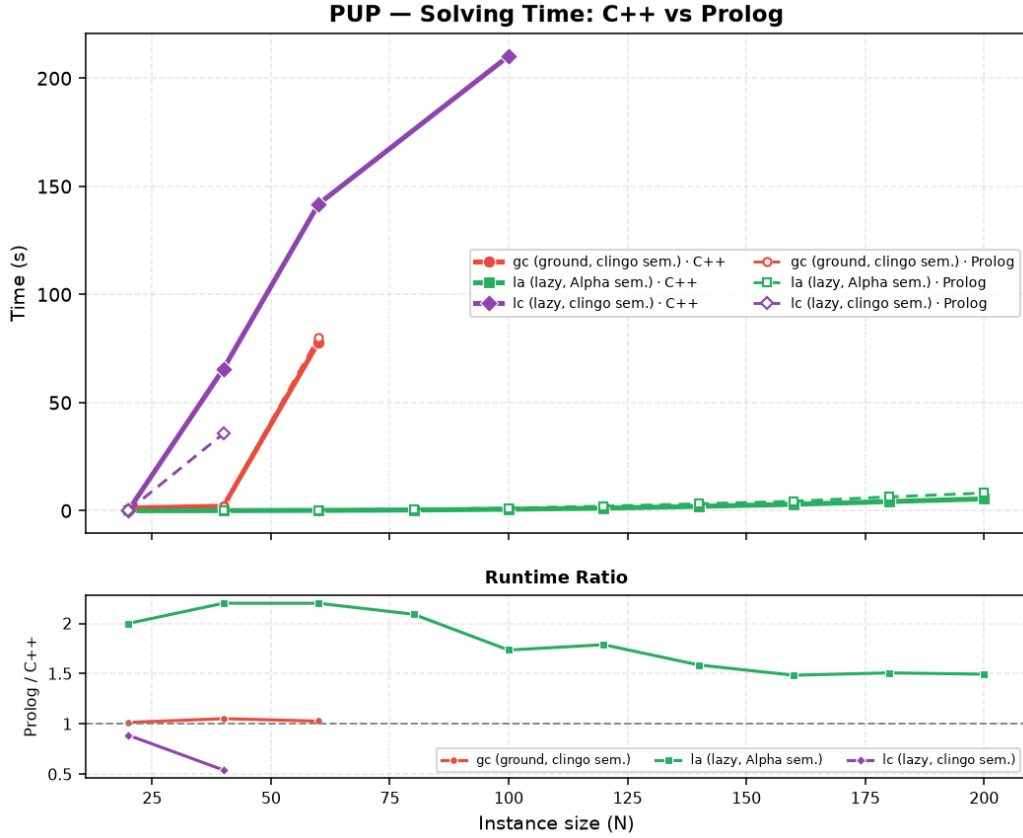


Figure 4.18: Solving time (top) and Prolog-to-C++ runtime ratio (bottom) on PUP across execution backends.

Figure 4.19 shows what the two effects add up to when the same variant runs on both backends. The total lazy overhead therefore factorizes as:

$$\text{Total Overhead} = \text{Decide Calls} \times (\text{Query Cost} + \text{Amortized Synchronization Cost}).$$

Lazy evaluation is thus cheapest exactly when the heuristic works: a richer condition costs more per decision, but a heuristic that guides well leaves few decisions to pay for.

4.10 Two Ground Simulations of Alpha: `ga` versus `ga_weak`

How closely can Alpha semantics be simulated within a ground-and-solve pipeline? Section 2.1.1 introduced two distinct transformations: `ga_weak`, which removes negative atoms from rule bodies, and `ga`, which replaces aggregate equality checks with dynamically unrolled monotone bounds ($S \leq \#\text{sum}\{Y : c(Y)\}$).

Neither transformation is a general-purpose rewriting, and the results below are to be read within the condition established in Section 2.1.1: deleting a negative literal preserves the Alpha reading only when the deleted atom is the target or its complement. BSP satisfies that condition, so on this family the two variants are

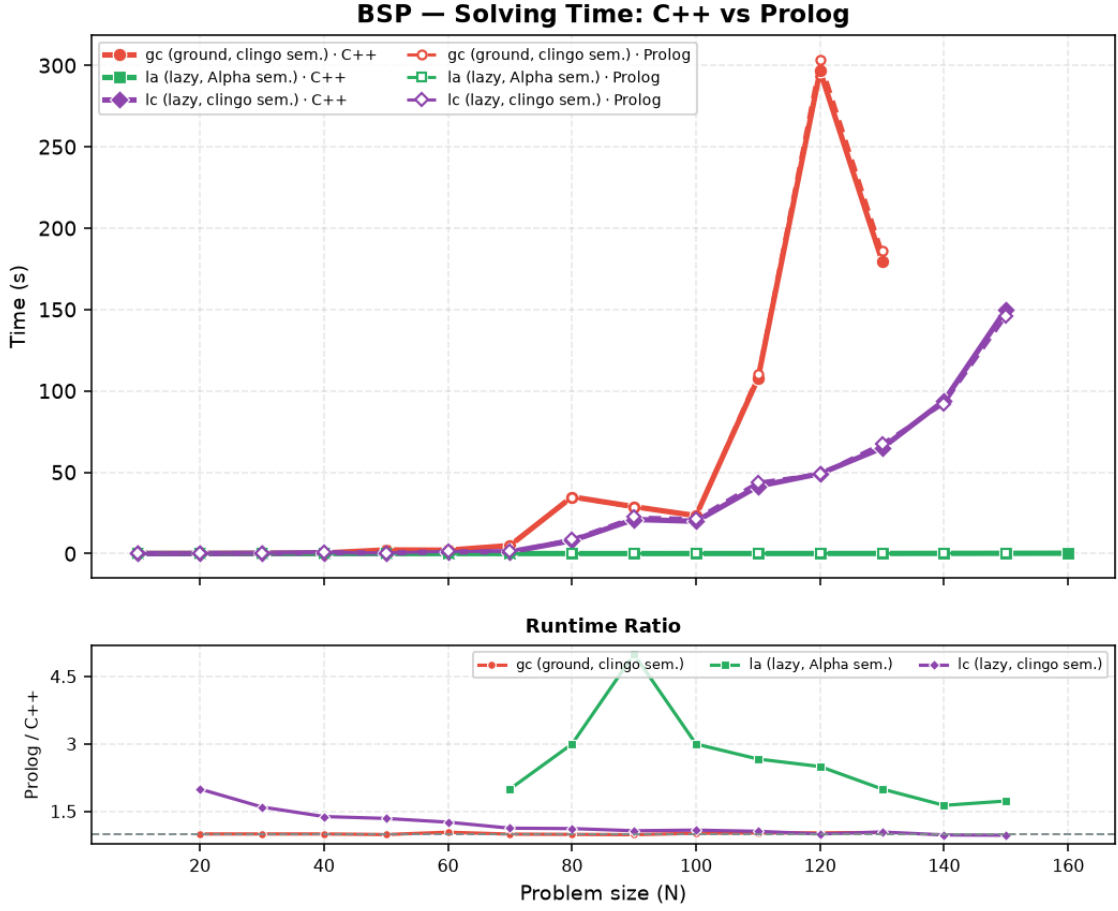


Figure 4.19: BSP: solving time comparison between the two backends. The bottom subplot shows the relative runtime ratio ($\text{Time}_{\text{Prolog}}/\text{Time}_{\text{C++}}$), where values above 1.0 indicate the factor by which the interpreted Prolog backend is slower than compiled C++.

faithful simulations and the comparison measures what it claims to. HRP does not, which is why the corresponding HRP discussion (Section 4.6) treats **ga** as an approximation rather than as a reference.

In BSP, both **ga** and **ga_weak** produce the exact same $\Theta(n^3)$ ground directives as the baseline **gc** (1,120 directives at $n = 10$, 8,440 at $n = 20$, and 27,960 at $n = 30$). Dropping the **sum_value(S)** domain in **ga_weak** does not reduce the grounding size: with the equality binding in place, the grounder must still enumerate all subset sums. The main difference is that equality bindings trigger at most once, and only after all atoms in the sum are assigned, whereas monotone bounds activate incrementally as the partial sum grows.

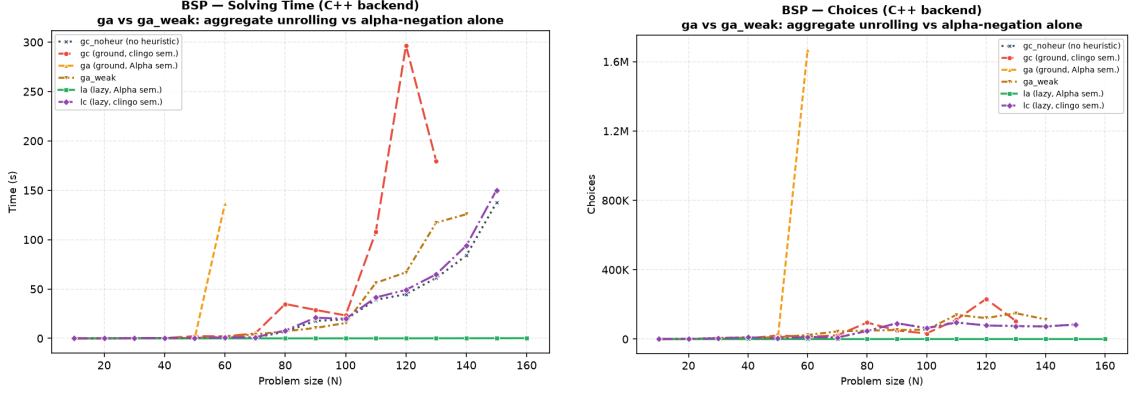


Figure 4.20: Solving time (left) and solver choices (right) for BSP comparing the Alpha-like semantics simulations (**ga** and **ga_weak**) against the baseline variants.

Beyond $n = 50$, **ga** deteriorates rapidly as weights exceed the `int16` range and saturate at 32,767: distinct partial sums collapse onto the same weight, and the ranking stops discriminating between candidate decisions (Figure 4.20). Up to $n = 50$ it matches **la** exactly, at n choices and zero conflicts; at $n = 60$ it already required 1.67 million choices and 136.5s of solving time, before timing out from $n = 70$ on. The limit is not the cost of the unrolling: at $n = 70$, the first size **ga** fails to solve, that unrolling is 0.6s of grounding and thirty-six megabytes over the lazy variant. It is the weight field, into which the reconstructed value of the aggregate has to fit.

ga_weak never hits the timeout, even though its weight expression is the same as **ga**’s: the equality binding is satisfied only once every element of the sum is assigned, so the heuristic almost never activates and the saturation is never reached. For the same reason it gives no useful guidance. At $n = 120$ its search trajectory is worse and noisier than the heuristic-free baseline, taking about 50% more choices and correspondingly longer to solve (Table 4.5).

Omitting the negative guards without unrolling the aggregate bounds therefore yields no usable guidance: it increases search effort and slightly reduces the solvable horizon. Read together with **ga**, the pair delimits the ground side of the design space from both ends: the faithful ground simulation works only while the weights fit, and the cheap approximation does not work at all.

4.11 Evaluating Pre-Materialized Auxiliary Predicates

The **la_aux** variant investigates whether the primary advantage of lazy evaluation stems simply from reducing the number of `__heuristic` directives, or from keeping the evaluation of heuristic conditions entirely outside the ground program. In **la_aux**, heuristic conditions and aggregate values are pre-materialized into auxiliary ASP predicates (`h_b(X,S)` and `h_c(X,S)`), which the lazy propagator then queries during search.

Defining auxiliary rules forces the grounder to instantiate the full Cartesian product of elements and partial sums (X, S) . As a consequence, solver variables increase

to 132,756 at $n = 50$, compared to just 5,150 in direct `1a`.

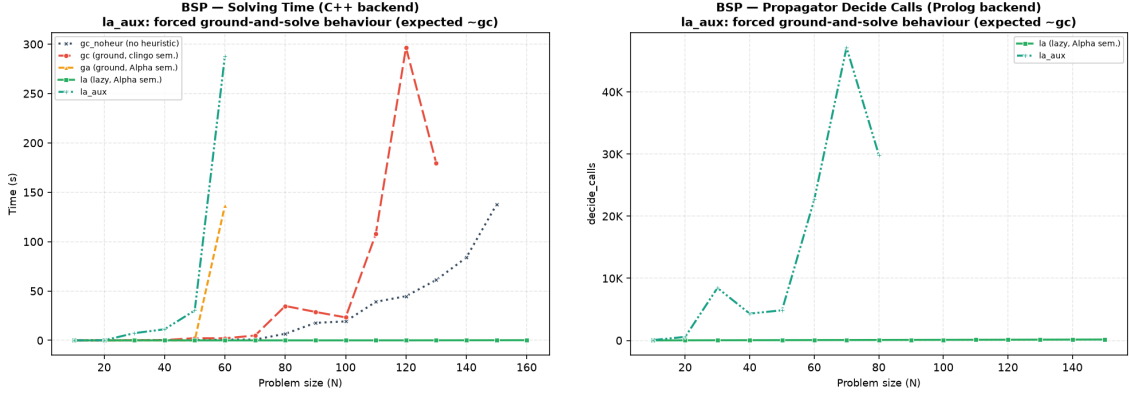


Figure 4.21: Solving time on the C++ backend (left) and number of propagator decide calls on the Prolog backend (right) comparing `la_aux` against direct `1a` in BSP.

The auxiliary materialization costs performance on both backends (Figure 4.21):

- Because the propagator must scan all materialized auxiliary candidates, the per-decision cost scales as $\mathcal{O}(n^2)$. Solving time diverges sharply from $n = 40$, reaching 288.1 s at $n = 60$ before consistently timing out from $n = 70$ onward. In contrast, the direct `1a` formulation maintains negligible solving times across the entire range.
- In the Prolog backend, the sheer volume of auxiliary candidates multiplies the required consultations, jumping from 573 calls at $n = 20$ to 8,452 at $n = 30$ (compared to exactly n calls in direct `1a`). This is the configuration that gains most from moving the trail out of the Prolog database: mirroring tens of thousands of auxiliary atoms per decision used to consume 99.2% of the run and capped the backend at $n = 30$, whereas reading them through foreign predicates leaves `la_aux` solving up to $n = 80$, two sizes beyond the native backend’s own frontier for this variant.

Reducing the number of heuristic directives is therefore not enough on its own: the benefit comes from keeping the evaluation of the heuristic conditions out of the ground program. Once those conditions are materialized as auxiliary rules, the combinatorial cost simply moves from the directives to the base encoding, and the lazy propagator inherits it at every decision.

4.12 Effect of the Constraint Formulation

Every result reported for BSP so far shares one confound: the base program is enormous, and it is the same for all variants. The `_co` family removes it. The transformation replaces the quadratic difference check over the two sums with a single constant-bound interval constraint (Section 3.3.1), and it is applied to all six variants of the matrix, so that each has a twin differing from it in the domain encoding alone. Two questions follow, and the family answers them separately: what

the reformulation does to the frontier, and, once the base program costs nothing, whether the heuristics still make any difference.

In the standard formulations (`gc`, `la`, and `lc`), this quadratic constraint dominates the grounding phase: at $n = 120$ it produces about 52.8 million propositional rules and required between two and three minutes of grounding. Direct lazy evaluation (`la`) removes the heuristic directives, but the ground program remains bound by the size of the domain specification.

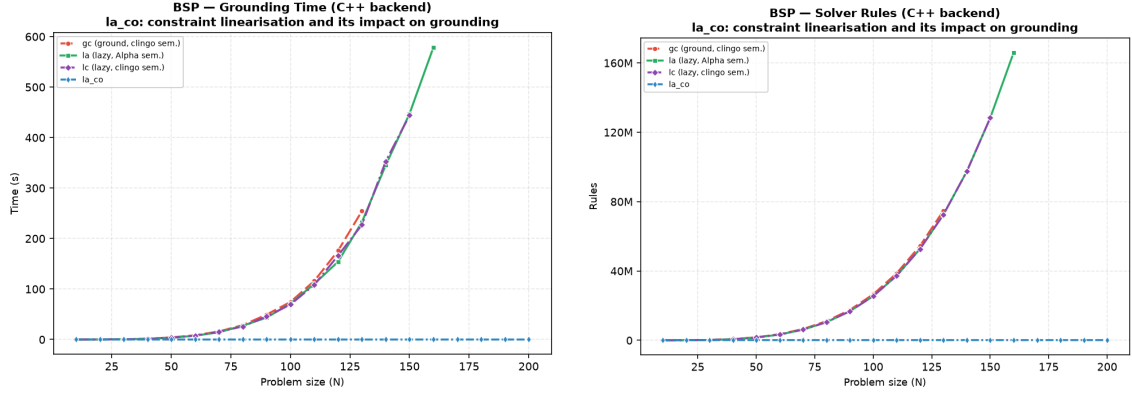


Figure 4.22: Grounding time (left) and propositional rule count (right) for BSP on the C++ backend, comparing the compact constraint formulation (`la_co`) against the baseline encodings (`gc`, `la`, and `lc`).

The reformulation changes formula size and grounding time by orders of magnitude (Figure 4.22):

- `gc`, `la`, and `lc` overlap near the top of both plots, sharing the same quadratic base encoding, whereas `la_co` flattens along the horizontal axis. At $n = 120$ the propositional formula drops from 52.8 million rules to 366.
- Grounding all but disappears. Total clingo runtime at $n = 120$ falls from 154.2s in `la` to 6ms, with the search guidance unchanged (117 choices, zero conflicts). The same holds up to $n = 200$, which is solved in 32ms of clingo time; the corresponding `runlim` wall time of 0.15s reported in Table 4.9 additionally includes process startup, and the two figures are not in conflict.

The two bottlenecks are therefore independent. Once lazy heuristics remove the search cost, what remains in BSP is set entirely by the materialization of the domain constraints, and pairing lazy heuristics with a compact encoding addresses both at once.

4.12.1 The full `_co` family: do the heuristics still matter?

Applying the same reformulation to every variant answers a question the sections above cannot: with the base program reduced to a few hundred rules, does the choice of heuristic still change anything? Table 4.11 reports the whole family at $n = 200$, the largest benchmarked size, which every one of the six variants now solves.

Variant	Rules	Variables	Grounding (s)	Total (s)	Peak RSS	Choices
gc_noheur_co	604	201	0.004	0.004	< 1 MB	60
gc_co	8,161,998	8,120,796	44.32	45.35	4,393 MB	100
ga_co	8,101,503	40,599	29.99	30.96	1,176 MB	114
ga_weak_co	8,161,998	80,796	26.39	27.03	839 MB	107
la_co	606	201	0.02	0.03	5 MB	197
lc_co	606	201	< 0.01	0.01	< 1 MB	60

Table 4.11: The `_co` family on BSP at $n = 200$, C++ backend. All six variants solve every size from 10 to 200, and none spends more than about a second in search at any of them. The heuristic is the only thing that differs between a row and its twin in Table 4.5.

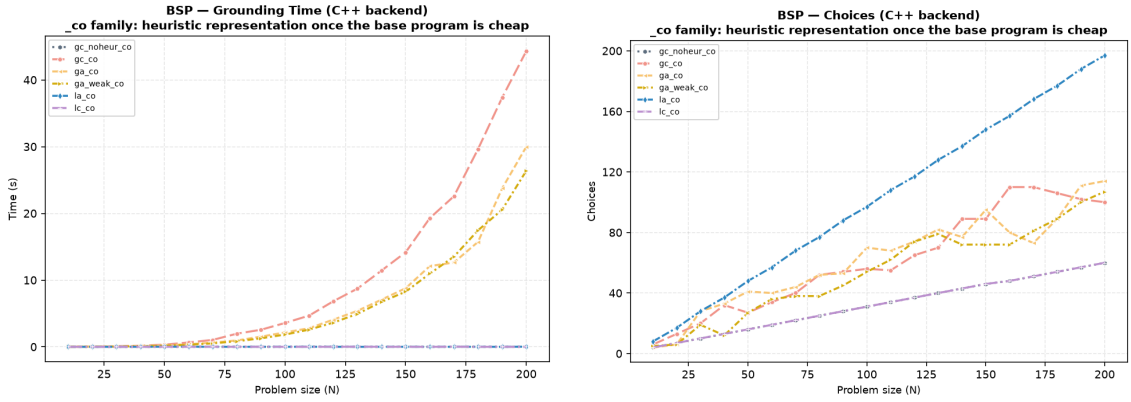


Figure 4.23: The `_co` family on BSP: grounding time (left) and solver choices (right). The ground and the lazy variants separate on grounding by three orders of magnitude; on choices all six lie within a small factor of each other, and the two lowest curves are the ones carrying no effective heuristic.

On search, the answer is no. Once the balance constraint is stated compactly, BSP stops being a search problem, and Figure 4.23 shows the two axes parting company: the grounding curves fan out over four orders of magnitude while the choice curves stay bunched together. The heuristic-free `gc_noheur_co` solves $n = 200$ in four milliseconds, taking 60 choices with zero conflicts, because the interval constraint propagates: fixing part of the partition determines the rest without a decision. No variant improves on that. `la_co`, which carries the guidance that on the standard encoding reduced search from 78,565 choices to 120, here takes 197, more than three times the baseline, since it insists on deciding every element along the greedy path where propagation would have settled most of them for free. The ground variants land in between, at 100 to 114. Search costs about a second at worst for any of them, so none of this is a performance difference; it is the observation that on this encoding there is no search left for a heuristic to improve.

This is worth stating plainly because it bounds the BSP results of Sections 4.3 and 4.8. The dramatic search separations reported there, three orders of magnitude between `la` and the baseline, are separations on a formulation whose difficulty is an

artifact of the quadratic constraint. They measure the guidance faithfully, but the thing being guided is a search that a better domain encoding does not have.

On representation, the answer is yes, and it is now measured cleanly.

The same table makes the opposite point on the other axis. With the base program at 604 rules, everything above that number *is* the heuristic, and the spread is the widest in the thesis. The two lazy variants ground 606 rules in twenty milliseconds and never exceed five megabytes. The three ground variants materialize between 8.10 and 8.16 million rules, spend 26 to 44 seconds doing so, and reach 0.8 to 4.4 GB. That is a factor of 13,500 in formula size and of three orders of magnitude in grounding time, attributable to the heuristic representation and to nothing else, on a benchmark where previously the base program contributed 52.8 million rules to every row alike. Section 4.2 counted directives; this measures what they cost once nothing else is in the way.

Two secondary findings. First, `lc_co` reproduces `gc_noheur_co` exactly, 60 choices and zero conflicts at every size, as `lc` reproduces `gc_noheur` on the standard encoding: the inertness of the clingo reading is a property of the heuristic body, not of the domain encoding it sits in. What the `_co` family adds is the price tag. Its ground twin `gc_co` buys the identical guidance, none, for 44.3s of grounding, 8.1 million rules and 4.4 GB, and ends up taking 100 choices where the baseline takes 60. The claim of Section 4.8, that an inert *materialized* heuristic is worse than no heuristic, holds here in its sharpest form.

Second, `ga_co` solves the entire range, including the sizes at which `ga` times out. Its weights saturate clasp’s 16-bit field exactly as `ga`’s do, since the two encodings differ only in the balance constraint and the weight expression is untouched, but the saturation costs nothing here, because the ranking it corrupts is not what is carrying the run. The limit identified in Section 2.1.4 is therefore a limit on the ranking rather than on the encoding: it bites exactly when the heuristic is doing the work, which on the standard BSP encoding it is and on the compact one it is not.

What the family establishes. Taken together, the two halves are the reason the lazy representation is worth having independently of the guidance it carries. On a compact encoding the heuristic buys no search, and a modeller might reasonably drop it; but if it is kept, because the same directives are wanted for a family of instances or because the encoding is not under the modeller’s control, then the ground form costs seconds and gigabytes to express something that the lazy form expresses in two facts and twenty milliseconds. The gap between `la_co` and `ga_co` is the whole of the mechanism this thesis implements, with every other variable held fixed.

4.13 C++ versus Prolog: Backend Comparison

Across the campaign the compiled C++ and interpreted Prolog backends behave consistently, producing identical solvability frontiers in all but four configurations.

Whenever heuristic consultations are infrequent or cheap, as in **1a** on PUP and on the whole `_co` family, both engines reach the same instance size.

Where the two diverge, the direction is not uniform, and the reason is the synchronization design of Section 4.9. In three cases the Prolog engine fails earlier: **1a** on BSP by a single step ($n = 160 \rightarrow 150$), because that run finishes within seconds of the time limit; and **1c** on PUP ($N = 100 \rightarrow 40$) and on HRP ($N = 16 \rightarrow 12$), where an unguided search triggers tens of thousands of futile queries and Prolog’s per-call cost becomes the bottleneck. HRP is the sharper of the two, and the only configuration in the campaign where the backend choice costs solved instances outright. In the fourth case the Prolog engine reaches *further* than the native one: the exploratory **1a_aux** on BSP now solves to $n = 80$ against the native backend’s $n = 60$, because that variant’s cost was almost entirely trail synchronization and reading the trail through foreign predicates removed it.

The C++ backend evaluates heuristic decisions two to six orders of magnitude faster than the interpreted Prolog engine ($\approx 10^{-4}$ ms against 0.07–72 ms per call), and the spread within that range is set by how many candidates the rule body enumerates rather than by the benchmark family. The native backend is therefore the faster of the two, and the Prolog backend the more expressive: its rule bodies are ordinary logic programs, which makes prototyping easier, and it accepts constructs the native fragment rejects outright (Section 2.3.3).

4.14 Summary of Results

The three tables below restate every claim this chapter makes against the measurement that supports it, and against the section where that measurement is reported: Table 4.12 for what lazy evaluation buys, Table 4.13 for what the clingo reading costs, and Table 4.14 for the price of the mechanism and the ablations that delimit it.

Claim	Evidence	Where
Lazy evaluation makes the heuristic representation constant in the instance size	1,010,200 ground directives $\rightarrow$ 2 template facts at $n = 100$ (BSP)	§4.2
... and removes it from the propositional formula	1,030,606 solver variables in gc $\rightarrow$ 20,300 in 1a , identical to the heuristic-free baseline	§4.4
Where the base program owns the frontier, the gain is in cost, not reach	BSP $n = 120$: solving 44.8 s $\rightarrow$ 0.04 s, 78,565 $\rightarrow$ 120 choices; frontier 150 $\rightarrow$ 160	§4.3
Where heuristic grounding is the binding constraint, the gain is a factor of two	PUP $N = 200$: grounding 430.1 $\rightarrow$ 218.5 s, peak RSS 24.4 $\rightarrow$ 12.7 GB, same instance solved	§4.5
With the base program removed, the representation gap is the only thing left	BSP _co at $n = 200$: 8.16 M rules and 44.3 s for gc_co against 606 rules and 0.02 s for 1a_co	§4.12.1
Dynamic guidance more than doubles the reach of the unguided baseline	PUP: gc_noheur fails at $N = 100$, 1a solves all ten instances	§4.5
The Alpha reading gives optimal guidance on BSP and HRP	BSP: exactly n choices, 0 conflicts at every n ; HRP: 122 choices, 0 conflicts at $N = 20$	§4.3, §4.6
Against the reference lazy solver, the propagator delivers the guidance and not the grounding	PUP: 4.5 $\times$ Alpha's time at equal memory, while ga is 8.7 $\times$ slower and uses 1.9 $\times$ its memory; BSP: identical trajectory	§4.7
Lazy grounding and dynamic heuristics are separable and neither suffices alone	alpha_noheur stops at $n = 40$ on BSP and solves no PUP instance	§4.7

Table 4.12: Part I: what lazy evaluation buys.

Claim	Evidence	Where
Under the clingo reading the heuristic is inert when every guard needs a complete assignment	BSP, PUP: zero applicable candidates per call; 1c trajectory identical to the baseline point by point	§4.8
... and misleading when only some guards do	HRP: 66,717 choices at $N = 14$ against 86 ($\approx 800\times$), 14.2 M at $N = 16$, timeout from $N = 18$	§4.8
An inert <i>ground</i> heuristic is worse than no heuristic	BSP: frontier 150 $\rightarrow$ 130; PUP: gc fails at $N = 80$, before the baseline; BSP _co : gc_co pays 44.3 s and 4.4 GB for the guidance 1c_co gets for nothing	§4.8, §4.12.1

Table 4.13: Part II: what the clingo reading costs.

Claim	Evidence	Where
The overhead is decisions $\times$ consultation cost, and vanishes when the heuristic works	0.03% of runtime (1a , BSP) to a timeout (1c , HRP)	§4.9
Reading the trail instead of mirroring it removes synchronization as a cost centre ... but shifts it onto rule bodies that enumerate	PUP $N = 200$: sync 17.5 $\rightarrow$ 0.63 s, level with the native backend's 0.68 s; the cost moves to inference (3.25 s) HRP $N = 20$: 1,248 candidates/call, 9.19 $\rightarrow$ 71.9 ms per call; 1a_aux gains five instances, 1c on HRP loses one	§4.9 §4.9, §4.13
The ground simulation of the Alpha reading is faithful only while the weights fit	ga = 1a up to $n = 50$; timeout from $n = 70$; clasp's ceiling is 32,767	§4.10
... and that ceiling binds the ranking, not the encoding	ga_co saturates identically yet solves all twenty sizes, because the compact encoding leaves no search to misrank	§4.12.1
Making only the negation dynamic buys nothing	ga_weak : 121,159 choices at $n = 120$, $\approx 50\%$ above the baseline	§4.10
The whole evaluation must stay out of the ground program	1a_aux : 132,756 variables at $n = 50$ against 5,150; timeout from $n = 70$	§4.11
Lazy heuristics and compact constraints remove the two bottlenecks together	1a_co : 52.8 M rules $\rightarrow$ 366 at $n = 120$; $n = 200$ solved in 32 ms	§4.12
On a compact encoding the heuristic buys no search, and the representation is all that separates the variants	gc_noheur_co takes 60 choices at $n = 200$, fewer than every guided variant; the ground ones still pay 26–44 s to say so	§4.12.1
The two backends agree on guidance and differ only in speed	Identical frontiers in all but four configurations; 10^{-4} ms against 0.07–72 ms per call	§4.13

Table 4.14: Part III: what it costs, and what it needs.

In prose, the chapter says three things. Moving dynamic aggregate evaluation from grounding to search reduces the heuristic representation from $\Theta(n^3)$ ground directives to two constant template facts, and where that component is what binds, as in PUP, this halves both grounding time and peak memory with respect to the static ground simulation of the same heuristic, while matching the reference lazy solver to within a small factor. Where the base program binds instead, as in BSP, the same reduction removes the search cost outright rather than moving the frontier, and pairing it with a compact domain encoding (**1a_co**) removes both bottlenecks at once.

The evaluation semantics decides whether any of that guidance works. The Alpha-like reading yields usable advice on partial assignments; the standard clingo reading leaves the heuristic inert on BSP and PUP and actively misleading on HRP, and materializing an inert heuristic into the ground program is worse than writing none.

Finally, laziness is paid for at every decision, and the price is the number of consultations times their cost. It is negligible exactly when the heuristic guides

well, since a good heuristic leaves few decisions to pay for, and it consumes an entire run when the guidance is wrong. The compiled backend pays two to six orders of magnitude less per consultation than the interpreted one, and the interpreted one buys, in exchange, a strictly larger fragment of the heuristic language. Within the interpreted backend the two terms of that product are separable, and moving the solver trail out of the Prolog database and behind foreign predicates removes the synchronization term outright, at the price of making the inference term depend on how much the rule body enumerates.

One boundary of all this is set by the domain encoding rather than by the mechanism. On BSP the `_co` family shows that a compact statement of the balance constraint leaves the solver with no search worth guiding, so the heuristic contributes nothing to the outcome, while the ground form of that same useless heuristic still costs eight million rules and forty seconds. The case for a constant-size heuristic representation is therefore strongest precisely where the case for the heuristic itself is weakest, which is not the argument this chapter set out to make.

Chapter 5

Discussion and Conclusions

5.1 Discussion of the Results

The chapter is organized around five findings, in the order in which the campaign produced them, with two engineering lessons attached to the ones they belong to.

The first finding is the one the thesis set out to test: an important part of the cost of declarative dynamic heuristics can be moved from grounding to search. On BSP, native `#heuristic` directives grow as $\Theta(n^3)$ and exceed one million ground objects at $n = 100$; the lazy variants keep two facts, and the difference is visible in the propositional instance size, in the peak memory, and in the timeout profile. The mechanism generalizes, and the full campaign measures the generalization. On PUP, whose heuristics multiply four unrolled value domains per ground pair, the ground Alpha simulation and the lazy variant both clear the entire published range under the 600-second/30 GB budget. The ground simulation, however, costs about twice the time and twice the memory of the lazy variant at every one of the ten common sizes, because the unrolled directives keep growing with the domains being unrolled. Both dominate the unguided baseline, which stalls by timeout at less than half their reach.

The two families therefore show the same mechanism from two sides. Where the heuristic grounding is the binding constraint, the lazy representation converts directly into a large cost advantage over the best ground heuristic, and into solved instances relative to the unguided baseline. Where the base program dominates, as at the top of the BSP range, it converts instead into an unchanged frontier reached with the search component reduced by three orders of magnitude; the residual wall there belongs to the domain encoding, not to the heuristic. The BSP frontier is unchanged rather than improved: the lazy variant, the clingo-like one and the heuristic-free baseline all stop within one size of each other, and the runs that decide the exact size finish within a few percent of the time limit. On that family the claim is about cost rather than reach, and the campaign’s design — structural counters alongside wall-clock times — is what keeps the two apart.

PUP is the family where the claim is about reach as well, but the shape of that claim needs stating carefully, because it is bounded by the benchmark rather than by the method. Against the unguided and the clingo-semantics baselines the separation is a frontier: they stall at $N \leq 100$ while the two Alpha-semantics variants

solve every published instance. Between those two variants, **ga** and **1a**, there is no separating instance, for the simple reason that **double-200** is the largest one the reference material contains and both solve it. What separates them there is cost at equal reach, and it is not small: 438.6s and 24.4GB against 224.0s and 12.7GB, on a 30GB budget that **ga** is within twenty percent of exhausting. Extrapolating the two curves puts **ga**’s ceiling around $N \approx 215$ and **1a**’s around $N \approx 280$ (Section 4.5), so the measured factor of two in memory should be worth roughly a third in instance size; but that is an extrapolation, it is reported as one, and turning it into a measurement needs instances that do not exist yet.

The external comparison of Section 4.7 puts a number on both halves of that sentence, and it is the most informative single experiment of the campaign because it is the only one whose yardstick is not clingo. Against Alpha in its dynamic-aggregate variant — the system whose semantics this thesis reproduces, running the authors’ own encodings — the lazy propagator delivers the guidance and not the grounding. On BSP it reproduces the reference trajectory exactly, n decisions and no backtracking, and then loses two orders of magnitude in wall time to a system that never builds the 52.8 million rules of the base program. On PUP, where the base program is proportionally less dominant, it lands within a small factor of the reference in time and at a peak memory the two systems cannot be told apart on, while the ground simulation of the very same heuristic is nearly an order of magnitude slower than Alpha and uses almost twice its memory; Section 4.7 gives the figures. Read as a whole, the ordering is **ga** < **1a** < Alpha: the lazy representation recovers most of the distance, and grounding accounts for the remainder. That is the claim the thesis makes, now measured against an external reference rather than asserted.

The same experiment also protects the semantic claim from an obvious objection, namely that the Alpha reading only looks good because it is being compared with clingo baselines. Alpha *without* its declarative heuristic stops at $n = 40$ on BSP, eleven sizes before ground clingo without a heuristic, and solves no PUP instance at all. Lazy grounding and dynamic heuristics are therefore separable contributions and neither is sufficient on its own: the four combinations of (eager, lazy) grounding and (absent, dynamic) heuristic land at $n = 150$, $n = 160$, $n = 40$ and $n = 200$ respectively. The effect this thesis isolates inside clingo is the same effect that makes the reference system usable.

This result should not be read as a replacement for general lazy grounding. clingo still grounds the base program; the propagator acts only on the heuristic component, and Section 4.7 measures exactly what that leaves on the table. Nor should it be read as a claim that laziness is free: the runtime work is real, it is paid at every decision, and the per-decision metrics exist precisely to keep that cost on the record. The honest formulation is a space–time trade-off with a crossover: below it, ground-and-solve remains competitive, because the explosion is manageable and the lazy machinery (in the Prolog case, an entire in-process logic-programming engine) carries a fixed overhead; beyond it, the constant-size heuristic component is the only representation that survives.

The second finding is semantic, and it is a finding about *meaning*, not performance. The same declarative heuristic produces opposite outcomes depending on

how partial information is read. Under the Alpha-like reading, the BSP heuristic is a perfect greedy strategy (n choices, zero conflicts) and the HRP policy reproduces its intended level structure decision for decision. Under the clingo-like reading, the very same weights and bodies degenerate in two distinct ways, which Section 4.8 documents together. On BSP and PUP the guards never open at decision time: the propagator reports zero applicable candidates per call, and the search is bit-identical to the baseline — a correspondence that survives the change of domain encoding, since `lc_co` likewise matches `gc_noheur_co` decision for decision. Call this inertness. On HRP the surviving low-level modifications steer the solver into a region almost eight hundred times larger at $N = 14$ and five orders of magnitude larger at $N = 16$, with both backends independently reaching the same order of magnitude of choices even though their exact counts differ. Call this misguidance.

Neither reading is “wrong”: they answer different questions, and the four-variant matrix exists to keep those questions separate. What the campaign shows is that the choice between them is not a tuning parameter but a precondition, because these encodings were written under one semantics and the other one cannot read their guards. A comparison that silently mixed the axes, for example by measuring `la` against `gc` and attributing the difference to laziness, would be methodologically invalid; the thesis takes some care never to do so. The clearest practical corollary is the one about the ground side: an inert heuristic is not a neutral heuristic if it has been materialized. On BSP the never-firing directives of `gc` cost two sizes of frontier against the baseline that carries no heuristic at all, $n = 130$ against $n = 150$, and on PUP they make `gc` fail before the baseline does. The lazy representation is therefore worth having even when the guidance is useless, which is not the argument the thesis set out to make and is arguably the more robust one.

The campaign adds a corollary about the ground simulation of the Alpha reading. `ga` reproduces the lazy guidance exactly while its weights remain representable, and stops working when they do not: on BSP that happens at $n = 51$, and from $n = 70$ the variant no longer finishes at all, making it the earliest-failing variant of the main matrix on BSP, nine sizes before the baseline. What stops it is not the price of the unrolling, which Section 4.10 measures and finds negligible against the base program; it is the field the reconstructed aggregate value has to fit into. Inside a ground-and-solve pipeline the lazy propagator is therefore not merely the cheaper carrier of the Alpha semantics; beyond a certain instance size it is the only one that can carry it at all, because it ranks its targets internally and never has to encode the aggregate as a weight.

The `_co` family sharpens that statement by supplying the case where the ceiling does no harm. `ga_co` runs into it on exactly the same weights and still clears every benchmarked size, because what the saturation destroys is the ordering among candidates, and an encoding whose propagation already determines the partition never consults that ordering. The ceiling is thus a limit on the ranking rather than on the encoding, and it becomes visible in proportion to how much the run depends on the heuristic — which is the same condition under which the heuristic is worth having in the first place.

The third finding generalizes that observation into a statement about the interface between the heuristic language and the solver that consumes it. clingo rep-

resents the weight of a `#heuristic` modification in a bounded field and saturates anything larger, so a weight is not an arbitrary integer but a resource with a ceiling. Any encoding that packs information into the weight, whether an Alpha level through the flattening rule of Section 2.1.3 or the value of an aggregate through the unrolling of `ga`, is spending that resource, and the two uses compete. Nothing warns the modeller when the ceiling is reached: the program still grounds, the run still terminates, the answer sets are still correct, and only the search degrades. The lazy propagator sidesteps the ceiling by construction, since it ranks targets internally and hands the solver a decision rather than a weight, and that is a design advantage worth stating separately from the grounding one.

The fourth finding concerns the translation layer between the two heuristic languages. Alpha’s weight–priority pairs are global levels; clingo’s priorities arbitrate on a single atom. Every encoding that expresses an ordering between different atom families must therefore flatten its levels into weights wherever clingo’s reading applies: in the ground variants, because clasp is the consumer, and in the lazy clingo variants, because the propagator deliberately reproduces clingo’s behaviour there. This is easy to state and easy to get wrong: an early revision of the lazy-clingo PUP and HRP encodings carried Alpha-style priorities that the clingo semantics does not rank globally, producing an unintended decision order that inverted the designed one (sensors before zones; cabinet-filling before reuse), and diverging from the Prolog twin encodings of the same variants. The final audit caught and corrected the discrepancy by applying the same flattening used in the ground encodings. The episode is instructive: the interaction between priority semantics and evaluation backend is exactly the kind of silent, non-crashing behavioural divergence that a benchmark pipeline cannot detect from exit codes, and it motivates the equivalence and wiring guards that the suite now includes.

The lesson was then turned into a structural decision rather than a matter of vigilance. As long as the priority field is used to carry levels in *some* encodings, its meaning depends on the pair (semantics, backend): a global level under one reading, a purely local arbiter under another, and a global level again in the Prolog backend, whose selection rule sorts by (priority, weight) regardless of the semantics selector. Three readings of one syntactic field is a standing invitation to the very bug just described. Since flattening is order-preserving, the suite removes the ambiguity by construction: the levels always live in the weight, in every variant and in both backends. The native heuristic language went one step further and dropped the priority construct altogether, so that a directive written for it cannot express a level except through its weight, and the parser rejects the field rather than silently reinterpreting it. Two properties follow. First, the encodings of a pair differ by exactly one token, so the axis under study is the only thing that varies. Second, the change is behaviour-preserving with respect to the earlier revision: on every benchmark the flattened weights reproduce the previous total order exactly, because on each instance the level multiplier strictly dominates the intra-level weights, so the experimental results reported here are unaffected.

That decision has a cost, and Section 2.3.3 states it: a fragment with no priority field cannot express a genuinely two-dimensional rank whenever the intra-level weight range is not known before grounding. The two backends consequently do not

cover the same language, the native one being a proper subset of the Prolog one, and the encodings of this suite were chosen to sit in the intersection so that the same experiment could run on both. That is a property of the benchmark selection and not of the mechanism, and it is the main thing a reader should not generalize from these results.

A related engineering lesson comes from the silent-fallback failure mode of the Prolog backend. A single missing environment variable degrades every lazy Prolog run to the no-heuristic baseline without any error, since the fallback evaluator simply fails on the Prolog-style rule bodies. The pipeline defends itself with positive evidence of activity (`decide_calls > 0`), cross-backend agreement checks, and a wrapper that forces the variable. Three of the episodes recorded in this chapter — the saturating weight field, the reinterpreted priority annotation, and this one — belong to the same class: the program grounds, the run terminates, the answer sets are correct, and only the guidance is silently gone. They argue for the same remedy, that a heuristic mechanism should check its own preconditions and prove it ran rather than merely not fail.

The fifth finding qualifies the first one, and it is the least comfortable of the set. On BSP the entire search advantage of the lazy variants rests on a domain encoding whose difficulty is self-inflicted. Restating the balance constraint compactly costs one line, and the resulting problem is solved by the unguided baseline in four milliseconds and sixty decisions at $n = 200$ — fewer decisions than any variant whose heuristic actually fires, and exactly as many as `lc_co`, whose heuristic never does. The heuristic, in other words, is not merely unnecessary on the compact encoding; it is very slightly counterproductive, because it insists on deciding elements that propagation would have fixed. A reader is entitled to conclude from this that the BSP heuristic was never the interesting thing about BSP.

What survives the observation, and is strengthened by it, is the representation claim. Table 4.11, which reports that four-millisecond baseline, also shows the three ground variants spending between 26 and 44 seconds and up to 4.4GB to express a heuristic that changes nothing, against twenty milliseconds and two facts for the lazy ones. With the base program reduced to six hundred rules there is no other explanation available for that spread: it is the heuristic component, measured on its own for the first time in the campaign. The practical reading is that the cost of *carrying* a declarative heuristic and the value of *having* one are independent quantities, and that the ground-and-solve pipeline couples them in the worst way, charging most for the heuristics it can least afford to unroll. That is an argument for the lazy representation which does not depend on the heuristic being good, and it is the one this thesis would keep if it had to keep only one.

Finally, the auxiliary-form experiment delimits the approach from within. `la_aux` shows that laziness only pays if the *entire* dynamic part of the heuristic, aggregate included, stays out of the grounder: materializing the heuristic body into an auxiliary relation reintroduces the very product the mechanism was built to avoid. This has a direct bearing on the expressiveness question above, because precompiling into auxiliary predicates is exactly the workaround by which the native fragment could accept a body it cannot express directly. The measurement says that this workaround is not a workaround: what the language cannot express,

the encoding cannot cheaply supply on its behalf.

5.2 Limitations

The current implementation has explicit technical limitations. The linguistic ones are not restated here: Section 2.3.3 gives the native fragment in full along its five axes — target, body, aggregate, weight and annotation — and the discussion above has already drawn the consequence, that the two backends do not carry the same language. Two limitations of the implementation itself are not linguistic and belong in this list instead: the propagator is registered sequentially and has no synchronization layer for parallel solving, and the tie-break between targets of equal rank is deterministic but does not reproduce clingo’s internal scores.

On the evaluation side, the campaign covers all three families on both backends with the corrected encodings, and both backends are phase-timed, but its scope needs stating precisely. Every cell of the design — one variant, on one instance, on one backend — is a single run, so the campaign measures no within-cell variance. Time-based claims are therefore reported as single measurements rather than averages. The separations this thesis rests on — the factor of two between `ga` and `la` on PUP, the two orders of magnitude of `lc` on HRP, the three orders of magnitude on the BSP solving component — are wide enough that run-to-run noise cannot account for them. The exception is the exact position of the BSP frontier, where the deciding runs finish within 6% of the time limit; repeated runs per cell would settle those two sizes, and remain the most valuable extension of the protocol.

The benchmark sets bound the conclusions in three further ways. The HRP instances remain easy for clasp at every benchmarked size, so HRP supports conclusions about guidance quality and per-decision cost, not about frontiers. On BSP the bound is sharper and is discussed above: the difficulty that the heuristic overcomes is a property of the quadratic balance constraint, and the `_co` family shows it dissolving under a one-line reformulation. Every BSP search result in this thesis should therefore be read as a measurement of guidance on a given encoding, not as evidence that the problem needs guidance. On PUP the claims rest on one instance per size in a single family, `double`, which ends at `double-200`: the non-monotone behaviour observed for `gc` and `lc` is a reminder that neighbouring sizes differ in hardness, per-size variance is not measured, and the comparison between `ga` and `la` is a comparison of cost on shared instances rather than of reach, because no available instance separates them. A further caveat concerns the Prolog backend specifically: its synchronization layer was rebuilt partway through the work, and the campaign reported here measures the current design throughout. The comparison against the previous one, quoted in Section 4.9, is therefore a comparison across two campaigns rather than within one, and while the effect is far too large to be noise — a factor of 27 on synchronization — the per-family totals it is embedded in also carry the ordinary run-to-run drift discussed above.

The two backends also do not reproduce identical search trajectories on PUP and HRP, where the heuristic ranking admits ties that the C++ and Prolog tie-breaks resolve differently. The divergence reaches fourteen percent on `la` and is larger still on the unstable `lc` searches, which is why the cross-backend comparisons in those

families are read qualitatively rather than as controlled measurements of the engine alone. The memory metric is end-to-end process RSS, which is the honest but blunt instrument discussed in Chapters 3 and 4: it includes the Prolog engine for the lazy Prolog variants and cannot separate grounding memory from search memory.

The external comparison with Alpha inherits three limitations of its own. It carries different weight on different families: on BSP and PUP it measures the mechanism this thesis is about, whereas on HRP, whose reference heuristic contains no dynamic aggregates, it measures the cost of lazy grounding on a program that does not need it, and the divergence observed there between Alpha’s two guided configurations is reported without being diagnosed. The two solvers’ internal counters are not commensurable, so the comparison rests on the two `runlim` measures alone. And Alpha’s peak RSS includes the JVM heap reserved through `-Xmx` rather than the heap in use, which makes its memory figures an upper bound, and the memory comparison on PUP a tie read conservatively rather than a measured equality.

Finally, the auxiliary clingo evaluator of the Prolog build re-grounds a program per decision and is a functional reference, not a performance-comparable backend, and the whitelist of static predicates together with the inference of the program constant n in the Prolog backend are benchmark-specific conveniences that a production version should replace with a declarative interface.

5.3 Implications

The main implication is that declarative heuristics can be treated as an intermediate layer between modelling and solving. One does not have to choose between fully grounded `#heuristic` directives and procedural heuristics hard-coded in the solver: a compact declarative surface, evaluated at runtime with access to the partial assignment, is a workable third point in the design space, and it can host more than one evaluation semantics behind one syntax. The two backends demonstrate two implementation strategies for that layer, a compiled, incremental one and an embedded general-purpose engine, with the expected trade-off between per-decision efficiency and expressive freedom of the rule bodies.

From an engineering perspective, the work also shows that clingo’s propagator and heuristic interfaces are flexible enough to prototype non-trivial guidance mechanisms without touching the solver core: watched literals, incremental aggregate states, per-target modifier resolution, and a global decision ranking are all expressible against the public API of `clingo::Heuristic`.

5.4 Future Work

Widening the native fragment. The single most valuable extension is the one Section 2.3.3 identifies: aggregates and body literals that can be joined to each other rather than only to the target tuple, together with comparison guards between bound values, integer division and modulo, and a principled treatment of non-numeric symbols through a stable internal mapping. The concrete target is the `doubleV` PUP family of [9], which the reference system runs unmodified and

which Section 2.3.3 shows to fall outside the native fragment on four independent counts. It is worth being explicit about why this is more than a convenience feature. `doubleV` is expressible today only for the Prolog backend; making it expressible for the native one is what would let the same experiment run on both backends, as every other encoding of this suite does, and `1a_aux` has already shown that the alternative — precompiling the missing joins into auxiliary ASP predicates — reintroduces precisely the grounding product the mechanism exists to avoid. A genuinely two-dimensional rank, restoring the priority field with the safeguards discussed earlier in this chapter, belongs to the same item, since `doubleV` also annotates a weight that is legitimately zero.

Indexing the foreign store for enumeration-heavy bodies. Removing the synchronization cost was future work in an earlier revision of this thesis and is now a completed change, reported in Section 4.9: `true_atom/1` and `false_atom/1` are registered as non-deterministic foreign predicates that read the assignment from the C++ side, recording a trail change is a hash-map update, and synchronization on `double-200` fell from 17.5s to 0.63s, level with the native backend. What that change also did was relocate the bottleneck, and the remaining work is where it went.

The residual cost is the snapshot described in Section 3.2.3: a generator call materializes the matching relation before it starts yielding. One snapshot per call is affordable when a rule body calls a generator a handful of times, as on PUP; it is not when the body joins several relations and yields over a thousand candidates per decision, as on HRP, where the per-call cost rose from 9.19ms to 71.9ms and `1c` lost an instance, `house-14`. Indexing by signature already keeps the enumeration proportional to the relation rather than to the trail, so what remains is the copy itself. Enumerating the store in place, with a cursor that survives across `PL_retry`, would remove it, and is the natural completion of the change — after which the Prolog backend should be uniformly cheaper than the design it replaced, rather than cheaper on two families and dearer on one.

Instances and benchmarks. Separating the `ga` and `1a` frontiers on PUP requires `double` instances larger than those published, around $N = 220$ and $N = 240$ on the extrapolation of Section 4.5. Since the ten reference files follow a regular construction, this means writing a generator and validating it by regenerating $N = 20 \dots 200$ and diffing byte for byte against the published set before trusting anything it produces beyond it; runs at that size also want a raised time limit, so that a failure of `ga` is unambiguously a memory failure. Independently of size, multiple instances per size would put variance bars behind the frontier claims, which the present one-run-per-cell protocol cannot support. Beyond PUP, configuration, scheduling, and packing domains are natural candidates for broadening the evaluation, precisely because their useful heuristics are dynamic. For HRP specifically, the instance set released with [18] is the obvious next step: it targets the same configuration domain and would extend the one family of this suite whose sizes are all easy for clasp, which is exactly the limitation recorded above.

Tooling and integration. A verified converter from annotated `#heuristic` directives to lazy facts, with the flattening rule of Section 2.1.3 applied mechanically rather than by hand, would remove the class of error the audit episode discussed earlier in this chapter exemplifies. The mechanism should also become configurable rather than always-on, and the backend should expose its per-decision statistics through clingo’s statistics interface instead of stderr parsing. A parallel-solving synchronization layer and a tie-break policy that cooperates with the solver’s own scores are the natural solver-side improvements.

5.5 Conclusions

This thesis presented a lazy evaluation mechanism for declarative domain-specific heuristics in clingo, instantiated in two backends over a common propagator design: a native C++ backend interpreting compact `__heuristic` facts with incremental dynamic aggregates, and an in-process SWI-Prolog backend evaluating heuristic rules as logic programs against the solver trail, which it reads in place through foreign predicates. The mechanism supports two explicit readings of partial information, the clingo-like and the Alpha-like semantics, and reproduces clingo’s weight, priority, and modifier behaviour where that reading applies, including the systematic flattening of Alpha’s global priority levels into clingo weights.

The full campaign on BSP, PUP, and HRP ran under uniform limits of 600 seconds and 30 GB. It reduces the ground heuristic component from $\Theta(n^3)$ directives to two constant facts, with the corresponding saving in real memory wherever the heuristic grounding matters. On PUP the lazy variant reaches more than twice the solvable frontier of the unguided baseline, and costs half the time and half the memory of the best ground heuristic at every size both solve. On BSP, where the base program owns the frontier, it removes the search cost outright — n choices and zero conflicts at every size, against tens of thousands for every alternative — and reaches the same wall as the baseline having spent 0.04 seconds of solving where the baseline spends 45. Restating the BSP balance constraint compactly, and running the whole matrix again on it, then separates the two halves of that result: the search advantage disappears, because the reformulated problem needs no guidance, while the representation advantage stands alone and unconfounded at eight million ground rules against six hundred.

The campaign also quantifies the price, on both backends: a per-decision runtime cost that ground-and-solve does not pay. It factorizes into consultation count times consultation cost, ranges from a twentieth of a percent of a run to the whole of it, and differs between the compiled and the interpreted evaluator by two to six orders of magnitude per consultation. It vanishes when the heuristic guides well and dominates when it does not. Within the interpreted backend the two terms proved separable in practice as well as on paper: replacing the mirrored trail with foreign predicates that read the solver’s assignment directly removed synchronization as a cost centre altogether, bringing it level with the compiled backend, and moved the residual cost onto rule bodies that enumerate many candidates. The suite validates that all variants and both backends preserve the answer sets, and guards against silent misconfiguration.

Measured against Alpha itself, running the authors' encodings, the lazy propagator reproduces the reference guidance and closes most of the distance to a native lazy-grounding solver: within a factor of 4.5 in time and at indistinguishable peak memory on PUP, where a ground simulation of the same heuristic is instead nearly nine times slower than the reference and uses almost twice its memory. The residual gap consists entirely of the base program that clingo still grounds.

The final result is a working, documented, and measurable prototype together with a methodology: two orthogonal axes that keep representation and semantics separate, a flattening rule that makes Alpha-style encodings meaningful under clingo's reading of priorities, and an experimental discipline that states what each number measures. It does not solve the grounding bottleneck in general, but it provides a concrete, verifiable path for bringing dynamic declarative heuristics into clingo without always paying the cost of their full ground materialization.

Bibliography

- [1] Michael Gelfond and Vladimir Lifschitz. “The Stable Model Semantics for Logic Programming”. In: *Proceedings of the Fifth International Conference and Symposium on Logic Programming*. MIT Press, 1988, pp. 1070–1080.
- [2] Torsten Schaub. *Answer Set Solving in Practice*. Potassco slide package, University of Potsdam. Version 1.22.0, 4 October 2024. 2024. URL: <https://teaching.potassco.org>.
- [3] Martin Gebser et al. *Potassco Guide*. Second edition, version 2.2.0. University of Potsdam. 2019. URL: <https://potassco.org>.
- [4] Francesco Calimeri et al. “ASP-Core-2 Input Language Format”. In: *Theory and Practice of Logic Programming* 20.2 (2020), pp. 294–309.
- [5] Martin Gebser et al. “Domain-Specific Heuristics in Answer Set Programming”. In: *Proceedings of the Twenty-Seventh AAAI Conference on Artificial Intelligence*. 2013, pp. 350–356.
- [6] Carmine Dodaro et al. “Combining Answer Set Programming and Domain Heuristics for Solving Hard Industrial Problems”. In: *Theory and Practice of Logic Programming* 16.5-6 (2016), pp. 653–669.
- [7] Antonius Weinzierl. “Blending Lazy-Grounding and CDNL Search for Answer-Set Solving”. In: *Logic Programming and Nonmonotonic Reasoning (LPNMR 2017)*. Vol. 10377. Lecture Notes in Computer Science. Springer, 2017, pp. 191–204.
- [8] Richard Comploi-Taupe et al. “Domain-Specific Heuristics in Answer Set Programming: A Declarative Non-Monotonic Approach”. In: *Journal of Artificial Intelligence Research* 76 (2023), pp. 59–114. DOI: 10.1613/jair.1.14091. URL: <https://jair.org/index.php/jair/article/view/14091>.
- [9] Richard Comploi-Taupe, Gerhard Friedrich, and Tilman Niestroj. “Dynamic Aggregates in Expressive ASP Heuristics for Configuration Problems”. In: *Proceedings of the 25th International Workshop on Configuration (ConfWS 2023)*. Vol. 3509. CEUR Workshop Proceedings. CEUR-WS.org, 2023, pp. 75–84. URL: <https://ceur-ws.org/Vol-3509/paper11.pdf>.
- [10] Martin Gebser et al. “clasp: A Conflict-Driven Answer Set Solver”. In: *Logic Programming and Nonmonotonic Reasoning (LPNMR 2007)*. Vol. 4483. Lecture Notes in Computer Science. Springer, 2007, pp. 260–265.
- [11] Matthew W. Moskewicz et al. “Chaff: Engineering an Efficient SAT Solver”. In: *Proceedings of the 38th Design Automation Conference*. 2001, pp. 530–535.

- [12] Martin Gebser et al. “Theory Solving Made Easy with Clingo 5”. In: *Technical Communications of the 32nd International Conference on Logic Programming (ICLP 2016)*. Vol. 52. OpenAccess Series in Informatics (OASICS). Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 2016, 2:1–2:15. DOI: 10.4230/OASICS.ICLP.2016.2.
- [13] Jan Wielemaker et al. “SWI-Prolog”. In: *Theory and Practice of Logic Programming* 12.1-2 (2012), pp. 67–96.
- [14] Markus Aschinger et al. “Optimization Methods for the Partner Units Problem”. In: *Integration of AI and OR Techniques in Constraint Programming (CPAIOR 2011)*. Vol. 6697. Lecture Notes in Computer Science. Springer, 2011, pp. 4–19.
- [15] Gerhard Friedrich et al. “(Re)configuration using Answer Set Programming”. In: *Proceedings of the IJCAI 2011 Workshop on Configuration*. Vol. 755. CEUR Workshop Proceedings. CEUR-WS.org, 2011, pp. 17–24.
- [16] Potassco Group. *benchmark-tool: A tool for benchmarking solvers and orchestrating cluster runs*. URL: <https://potassco.org/benchmark-tool/>.
- [17] Armin Biere. *runlim: a tool for sampling time and memory usage of a process*. URL: <https://github.com/arminbiere/runlim>.
- [18] Veronika Semmelrock et al. *Learning to Solve and Optimize by Evolving Code*. 2026. arXiv: 2605.31049 [cs.LG]. URL: <https://arxiv.org/abs/2605.31049>.